UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

JOÃO VICTOR DOS SANTOS GOMES

META-APRENDIZADO PARA AVALIAÇÃO DA QUALIDADE DO PRÉ-PROCESSAMENTO E RECOMENDAÇÃO OTIMIZADA DE PIPELINES EM AUTOML

MEDIANEIRA

2026

JOÃO VICTOR DOS SANTOS GOMES

META-APRENDIZADO PARA AVALIAÇÃO DA QUALIDADE DO PRÉ-PROCESSAMENTO E RECOMENDAÇÃO OTIMIZADA DE PIPELINES EM AUTOML

Meta-learning for Preprocessing Quality Assessment and Optimized Pipeline Recommendation in AutoML

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Ciência da Computação do Curso de Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Orientador: Prof. Dr. Jorge Aikes Junior

Coorientador: Prof. Dr. Davi Pereira dos Santos

MEDIANEIRA

2026

4.0 Internacional

Esta licença permite remixe, adaptação e criação a partir do trabalho, para fins não comerciais, desde que sejam atribuídos créditos ao(s) autor(es) e que licenciem as novas criações sob termos idênticos. Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

JOÃO VICTOR DOS SANTOS GOMES

META-APRENDIZADO PARA AVALIAÇÃO DA QUALIDADE DO PRÉ-PROCESSAMENTO E RECOMENDAÇÃO OTIMIZADA DE PIPELINES EM AUTOML

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Ciência da Computação do Curso de Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Prof. Dr. Jorge Aikes Junior
Orientador
Universidade Tecnológica Federal do Paraná

Prof. Dr. Davi Pereira dos Santos
Coorientador
Universidade Tecnológica Federal do Paraná

Prof. Dr. Jandrei Sartori Spancerski
Membro da banca examinadora
Universidade Tecnológica Federal do Paraná

MEDIANEIRA

2026

AGRADECIMENTOS

Agradeço ao meu orientador, Prof. Dr. Jorge Aikes Junior, e ao meu coorientador, Prof. Dr. Davi Pereira dos Santos, pela dedicação, pela disponibilidade e pelos direcionamentos que tornaram possível a realização deste trabalho.

Agradeço aos meus pais, pelo apoio incondicional e pelo incentivo constante ao longo de toda a minha trajetória acadêmica.

Agradeço também aos amigos que estiveram comigo ao longo dos anos de faculdade, compartilhando as dificuldades e as conquistas dessa caminhada.

Por fim, agradeço à Universidade Tecnológica Federal do Paraná (UTFPR), Câmpus Medianeira, pela estrutura e pelo apoio institucional oferecidos durante o desenvolvimento deste trabalho.

RESUMO

A seleção de pipelines de classificação depende das características dos dados e das etapas de pré-processamento. Este trabalho investiga seu ranqueamento por meta-aprendizado com representação estatística determinística de conjuntos rotulados. O acervo histórico contém 201 tarefas OpenML, 60 pipelines e vetores de 64 posições, compostos por 55 descritores e nove zeros. A auditoria identificou tarefas relacionadas e oito casos incompatíveis com o mínimo de cinco exemplos por classe. A avaliação principal usa 193 tarefas em 165 grupos, cinco partições externas e ajuste de vizinhança em validação interna por grupos. Comparam-se k-NN multi-saída, Random Forest, média global e ranking aleatório usando Spearman, Kendall, Kendall ponderado, NDCG e regret. O Kendall ponderado médio por grupo foi $0{,}3995$ para k-NN, $0{,}4309$ para Random Forest e $0{,}3690$ para média global. A diferença pareada Random Forest menos k-NN foi $0{,}0314$, com intervalo descritivo de 95% $[0{,}0073; 0{,}0564]$; a vantagem do k-NN sobre a média global permaneceu incerta. O diagnóstico encontrou 17 tarefas com correlação ponderada não positiva e uma indefinida. Um piloto separado realizou 180 avaliações de hiperparâmetros em dados brutos com tratamento dentro dos folds. Os resultados delimitam a contribuição dos metamodelos neste benchmark, preservando como limitações a origem dos escores históricos, a avaliação retrospectiva e a ausência de reconstrução completa da Fase 1.

Palavras-chave: meta-aprendizado; automl; recomendação de pipelines; ranqueamento; embeddings de datasets.

ABSTRACT

Classification pipeline selection depends on dataset characteristics and preprocessing choices. This work investigates pipeline ranking through meta-learning with a deterministic statistical representation of labeled datasets. The historical archive contains 201 OpenML tasks, 60 pipelines and 64-position vectors comprising 55 descriptors and nine padded zeros. An audit identified related tasks and eight cases that failed a minimum of five observations per class. The primary evaluation uses 193 tasks in 165 groups, five outer partitions and group-based inner validation for neighborhood selection. Multi-output k-NN and Random Forest are compared with global-mean and random-ranking baselines using Spearman, Kendall, weighted Kendall, NDCG and regret. Group-averaged weighted Kendall was 0.3995 for k-NN, 0.4309 for Random Forest and 0.3690 for the global mean. The paired Random Forest minus k-NN difference was 0.0314, with a descriptive 95% interval of $[0.0073; 0.0564]$; the k-NN advantage over the global mean remained uncertain. Error analysis found 17 tasks with nonpositive weighted correlation and one undefined case. A separate hyperparameter pilot performed 180 evaluations on raw data with preprocessing fitted within folds. The results characterize predictor effects in this benchmark while acknowledging the limitations of historical performance scores, retrospective evaluation and the absence of a complete Phase 1 reconstruction.

Keywords: meta-learning; automl; pipeline recommendation; ranking; dataset embeddings.

LISTA DE FIGURAS

Figura 1 – Fluxograma do processo experimental: Fase 1 - Construção do meta-conjunto **30**
Figura 2 – Fluxograma do processo experimental: Fase 2 - Meta-aprendizado e recomendação no processo experimental **31**
Figura 3 – Kendall ponderado por modelo com intervalos descritivos por grupo . **40**
Figura 4 – Diagnóstico do k-NN: dimensionalidade, balanceamento e distância ao treino **40**

LISTA DE TABELAS

Tabela 2 – Comparativo entre trabalhos relacionados e a proposta deste TCC . . . 25

Tabela 3 – Exemplos de conjuntos de dados utilizados no meta-conjunto 29

Tabela 4 – Avaliação revisada: médias por grupo em cinco partições externas . . 39

LISTA DE QUADROS

Quadro 1 – Materiais, Ferramentas e Conjuntos de Dados Utilizados 27

LISTA DE ABREVIATURAS E SIGLAS

Siglas

ANOVA	*Analysis of Variance*
AutoML	*Automated Machine Learning*
BADGE	*Batch Active learning by Diverse Gradient Embeddings*
CIFAR-10	*Canadian Institute for Advanced Research 10 classes*
CIFAR-100	*Canadian Institute for Advanced Research 100 classes*
D3M	*Data-Driven Discovery of Models*
DAG	*Directed Acyclic Graph*
DARPA	*Defense Advanced Research Projects Agency*
DBTC	*Decision Boundary Topological Complexity*
DT	*Decision Tree*
FashionMNIST	*Fashion Modified National Institute of Standards and Technology database*
ISIC2019	*International Skin Imaging Collaboration 2019 Dataset*
k-NN	*k-Nearest Neighbors*
LOO	*Leave-One-Out*
LSTM	*Long Short-Term Memory*
MEGA	*Meta-learning Guided Algorithm Recommendation*
ML	*Machine Learning*
MLP	*Multilayer Perceptron*
MNIST	*Modified National Institute of Standards and Technology database*
NDCG	*Normalized Discounted Cumulative Gain*
PCA	*Principal Component Analysis*
RF	*Random Forest*
SVM	*Support Vector Machine*
TCM	*TypiClust-Margin*
UCI	*University of California, Irvine*

SUMÁRIO

1 Introdução 11
1.1 Objetivos 12
1.2 Justificativa 12
1.3 Estrutura do Trabalho 13
2 Referencial Teórico 15
2.1 Aprendizado de Máquina e AutoML 15
2.2 Meta-aprendizado 16
2.3 Meta-características e Caracterização de Conjuntos de Dados 17
2.4 Aprendizado Ativo 17
2.5 Seleção e estrutura de *pipelines* 18
2.6 Componentes do Arsenal Experimental 19
2.7 Embeddings de Conjuntos de Dados e Métricas de Ranking 20
2.7.1 Embeddings de Conjuntos de Dados 20
2.7.2 Métricas de Avaliação de Ranking 21
2.8 Trabalhos Relacionados 22
2.8.1 Representação de Conjuntos de Dados e Meta-características 22
2.8.2 Sistemas de Recomendação de Algoritmos e Pipelines 23
2.8.3 Avaliação de Fronteiras de Decisão e Aprendizado Ativo 24
2.8.4 Síntese Comparativa 25
2.9 Avaliação Contemporânea de AutoML e Meta-aprendizado 26
3 Materiais e Métodos 27
3.1 Materiais Utilizados 27
3.1.1 Linguagens e Ambientes de Desenvolvimento 28
3.1.2 Bibliotecas e Frameworks 28
3.1.3 Conjuntos de Dados 28
3.1.4 Recursos Computacionais 29
3.2 Métodos 30
3.2.1 Fase 1 – Construção do Meta-conjunto 31
3.2.2 Fase 2 – Meta-aprendizado por Ranqueamento 33
3.2.2.1 Preditor k-NN Multi-saída 33

3.2.2.2 Validação por Leave-One-Out 34
3.2.2.3 Métricas de Avaliação de Ranking 34
3.2.2.4 Análise de Correlação Estatística 35
3.2.3 Auditoria e Validação por Grupos 35
3.2.4 Arsenal de Pipelines e Piloto de Hiperparâmetros 36
3.2.5 Reprodutibilidade e Limites da Fase 1 36
4 Experimentos e Resultados 38
4.1 Auditoria do Meta-conjunto 38
4.2 Reprodução dos Resultados Históricos 38
4.3 Comparação em Validação por Grupos 39
4.4 Diagnóstico de Erros e Variabilidade 39
4.5 Piloto de Variação de Hiperparâmetros 40
4.6 Interpretação e Limitações 41
5 Conclusão 42
5.1 Síntese dos Resultados 42
5.2 Atendimento dos Objetivos 42
5.3 Limitações e Perspectivas 43
REFERÊNCIAS 44

1 INTRODUÇÃO

A seleção do algoritmo mais adequado para um determinado problema de aprendizado de máquina (ML, *Machine Learning*) é, reconhecidamente, um dos maiores desafios enfrentados por profissionais da área. Isso ocorre porque não existe um algoritmo universalmente superior para todos os tipos de dados, conforme reforçado pelo Teorema *No Free Lunch* Wolpert e Macready (1997), que demonstra formalmente que, em média sobre todos os problemas possíveis, qualquer dois algoritmos de otimização ou aprendizado têm desempenho equivalente — implicando que a superioridade de um algoritmo em um domínio necessariamente vem acompanhada de inferioridade em outro. As características intrínsecas dos dados — como dimensionalidade, balanceamento de classes, distribuição e presença de ruídos — impactam diretamente o desempenho dos modelos, tornando inviável a adoção de uma abordagem única e generalista.

Nesse cenário, o desenvolvimento de pipelines eficientes em ML exige uma combinação cuidadosa de três componentes fundamentais: representação, avaliação e otimização, conforme proposto por Domingos (2017). A representação define o espaço de hipóteses possível; a avaliação, os critérios que distinguem bons modelos de ruins; e a otimização, os métodos de busca que encontram o modelo mais adequado. Tradicionalmente, a construção desses pipelines depende fortemente da expertise humana, que realiza sucessivas tentativas e ajustes manuais.

Com o objetivo de reduzir esse esforço, surgem as abordagens de *AutoML* (*Automated Machine Learning*), que automatizam desde a seleção de algoritmos até o ajuste de hiperparâmetros. Deste modo, como destacam Schoenfeld (2020) e Vanschoren (2018), a eficácia dos sistemas de AutoML se amplia quando são utilizados modelos de meta-aprendizado, capazes de aprender a partir da experiência acumulada em múltiplos problemas anteriores.

O meta-aprendizado, ou aprendizado para aprender, estrutura-se na coleta de metadados e na extração de *meta-características* dos conjuntos de dados — atributos que capturam aspectos estatísticos, estruturais e informacionais dos dados Schoenfeld, Seppi e Giraud-Carrier (2022). Com isso, torna-se possível treinar metamodelos capazes de prever, de maneira informada, quais algoritmos ou pipelines possuem maior chance de sucesso em um novo problema.

Entretanto, apesar dos avanços, boa parte dos sistemas de AutoML e meta-aprendizado concentra-se na escolha do modelo e na configuração dos hiperparâmetros, desconsiderando um fator crítico: a qualidade do *pré-processamento dos dados*. Dados inconsistentes, com escalas divergentes, alta presença de *outliers* ou problemas de codificação categórica podem comprometer todo o pipeline, independentemente da qualidade do algoritmo selecionado.

Diante desse cenário, este trabalho propõe e avalia uma abordagem de meta-aprendizado para **ranqueamento e recomendação de pipelines** de classificação. A proposta consiste em: (i) construir um meta-conjunto a partir da execução sistemática de 60 pipelines — combinações de escaladores, seletores de atributos e classificadores — sobre um acervo histórico de 201 tarefas obtidas do OpenML (VANSCHOREN *et al.*, 2014), representando cada

conjunto por uma representação estatística distribucional de 64 dimensões, também referida neste trabalho como *embedding* estatístico por ser um vetor numérico de dimensão fixa que resume propriedades do conjunto de dados sem treinamento neural[1]; e (ii) avaliar preditores multi-saída k-NN e Random Forest, com reprodução LOO histórica e validação principal por grupos em 193 tarefas elegíveis, para estimar o ranking dos pipelines. Separar grupos e ajustar k no treino reduz a reutilização de tarefas relacionadas entre treino e teste.

A avaliação de recomendações exige distinguir representação, preditor e benchmark. A revisão de Baratchi *et al.* (2024) organiza AutoML em espaço de busca, estratégia e avaliação; essa distinção motiva controlar o arsenal ao comparar metamodelos.

1.1 Objetivos

Este trabalho tem como objetivo geral desenvolver e avaliar um sistema de meta-aprendizado baseado em embeddings estatísticos de conjuntos de dados para ranqueamento e recomendação de pipelines de classificação, capaz de identificar, a partir das características do conjunto de dados de entrada, quais configurações de pré-processamento e classificação apresentam maior probabilidade de sucesso preditivo. Para alcançar esse objetivo geral, foram definidos os seguintes objetivos específicos:

- Avaliar, por meio da construção de um meta-conjunto — repositório de conhecimento contendo descrições de pipelines, embeddings dos conjuntos de dados e rótulos de desempenho obtidos por validação cruzada estratificada —, se metadados de desempenho e meta-características de conjuntos de dados carregam sinal suficiente para diferenciar boas e más combinações de pré-processamento e classificação;
- Construir e avaliar um modelo preditivo capaz de estimar, a partir de metacaracterísticas dos dados, o ranking de adequação de diferentes configurações de pré-processamento e classificação.

1.2 Justificativa

A relevância deste trabalho assenta-se em duas lacunas complementares dos sistemas atuais de AutoML. A primeira é a tendência de focar exclusivamente na seleção do algoritmo ou na otimização de hiperparâmetros, desconsiderando a avaliação conjunta das etapas de pré-processamento — escalonamento, seleção de atributos e classificação — como um pipeline integrado Schoenfeld, Seppi e Giraud-Carrier (2022). Problemas como escalonamento inadequado ou seleção de atributos prejudicial podem comprometer completamente o desempenho do modelo, independentemente da qualidade do classificador escolhido Domingos (2017).

[1] O conceito de *embedding* de conjuntos de dados e a distinção entre representações neurais e estatísticas são detalhados na Seção 2.7.

A segunda lacuna refere-se à forma de avaliação: a maioria dos sistemas de AutoML trata a recomendação como seleção do *melhor* pipeline (top-1), perdendo a informação ordinal completa. Schoenfeld, Seppi e Giraud-Carrier (2022) demonstraram que ranquear pipelines com base na similaridade de meta-características dos conjuntos de dados produz recomendações superiores às estimativas de desempenho absoluto. Contudo, as métricas de avaliação utilizadas na literatura raramente penalizam de forma diferenciada os erros nas primeiras posições do ranking — exatamente onde o impacto prático é maior para um usuário de AutoML que avalia os primeiros pipelines sugeridos.

Este trabalho preenche essas lacunas ao: (i) avaliar pipelines como combinações completas de pré-processamento e classificação; (ii) adotar ranqueamento como paradigma de recomendação; e (iii) utilizar Kendall Tau ponderado e NDCG como métricas complementares para avaliar a qualidade das primeiras posições do ranking, onde o impacto prático da recomendação é maior. O meta-conjunto construído, com 201 conjuntos de dados e 60 pipelines avaliados por validação cruzada estratificada, constitui um acervo reutilizável cuja auditoria e reavaliação integram a contribuição. Não se infere superioridade por escala ou métricas de protocolos distintos.

1.3 Estrutura do Trabalho

Este trabalho está organizado em cinco capítulos:

- **Capítulo 1 – Introdução:** apresenta o contexto, o problema de pesquisa, os objetivos, a justificativa e a estrutura geral do trabalho;
- **Capítulo 2 – Referencial Teórico:** discute os fundamentos de aprendizado de máquina, AutoML, meta-aprendizado, meta-características para caracterização de conjuntos de dados, aprendizado ativo, estrutura de pipelines, componentes de pré-processamento e classificação, embeddings de conjuntos de dados e métricas de ranqueamento, e revisa os principais trabalhos relacionados nas áreas de representação de conjuntos de dados por embeddings, sistemas de recomendação de algoritmos e pipelines via meta-aprendizado, e análise de complexidade de fronteiras de decisão;
- **Capítulo 3 – Materiais e Métodos:** descreve os materiais utilizados, a metodologia experimental em duas fases — construção do meta-conjunto e meta-aprendizado por ranqueamento —, os preditores k-NN e Random Forest, a auditoria, a validação por grupos e as métricas de ranking;
- **Capítulo 4 – Experimentos e Resultados:** apresenta a auditoria do acervo, a reprodução histórica, a comparação de metamodelos por grupos, o diagnóstico de erros e um piloto de hiperparâmetros em dados brutos;

- **Capítulo 5 – Conclusão:** sintetiza as contribuições do trabalho, verifica o atendimento dos objetivos, discute limitações identificadas e propõe direções para pesquisas futuras.

2 REFERENCIAL TEÓRICO

O aprendizado de máquina moderno envolve uma série de decisões interdependentes que impactam diretamente a eficácia dos modelos, desde o pré-processamento dos dados até a escolha e configuração dos algoritmos. A crescente complexidade desses processos impulsionou o surgimento do *AutoML*, cujo objetivo é automatizar etapas críticas de um pipeline preditivo. No entanto, para que essa automação seja eficiente, é necessário compreender os fundamentos que orientam tais decisões.

Este capítulo apresenta os principais conceitos que embasam a proposta deste trabalho, abordando inicialmente os fundamentos do Aprendizado de Máquina e a estrutura dos sistemas de *AutoML*. Em seguida, discute-se o meta-aprendizado como estratégia para reutilizar conhecimento adquirido em problemas anteriores, otimizando escolhas futuras. Também são exploradas as meta-características, essenciais para caracterizar conjuntos de dados e treinar metamodelos eficazes, bem como o aprendizado ativo, que visa otimizar o processo de rotulagem. Por fim, são apresentadas estratégias para seleção e estruturação de pipelines, os componentes de pré-processamento e classificação utilizados no experimento, as representações vetoriais de conjuntos de dados e as métricas de ranqueamento adotadas na avaliação.

2.1 Aprendizado de Máquina e AutoML

A aprendizagem de máquina envolve sistemas que aprendem automaticamente a partir de dados, sem necessidade de programação explícita do modelo. Domingos (2017) destaca que, para não se perder na variedade de algoritmos disponíveis, deve-se encarar o aprendizado como a combinação de três componentes principais: representação, avaliação e otimização. Nesse contexto:

- **Representação:** define a linguagem formal dos classificadores que o sistema pode aprender, isto é, o espaço de hipóteses permitido. Se um classificador não está nesse espaço, ele não pode ser obtido pelo aprendizado;
- **Avaliação:** função objetivo utilizada para distinguir classificadores bons de maus (por exemplo, acurácia, erro quadrático). Em geral, há uma função interna de otimização distinta da métrica externa desejada;
- **Otimização:** método de busca no espaço de hipóteses para encontrar o classificador de maior pontuação. Técnicas comuns incluem busca combinatória, algoritmos evolucionários, gradiente etc.

A automação desses componentes é o foco do AutoML. Sistemas de AutoML têm se mostrado eficazes ao empregar *metamodelos* — modelos treinados sobre o histórico de desempenho de algoritmos em execuções anteriores, e não diretamente sobre os dados do problema-

alvo — de forma *offline* (isto é, previamente, a partir de execuções já realizadas em outras bases) para guiar a seleção de algoritmos e parâmetros, abordagem detalhada na Seção 2.2. Em particular, Schoenfeld (2020) observa que sistemas AutoML obtêm melhor desempenho usando metamodelos offline para estimar o desempenho de modelos. Vanschoren (2018) aponta também que o meta-aprendizado pode acelerar e aprimorar o projeto de pipelines de ML, substituindo abordagens manuais por soluções aprendidas de forma orientada por dados.

2.2 Meta-aprendizado

O meta-aprendizado, ou "aprender a aprender"Vilalta e Drissi (2002), é a ciência de observar sistematicamente como diferentes abordagens de aprendizado de máquina performam em múltiplas tarefas, e então aprender com essa experiência (metadados) para resolver novas tarefas mais rapidamente, de modo a construir sistemas de aprendizado autoadaptativos que melhoram seu viés indutivo dinamicamente com a experiência acumulada Brazdil *et al.* (2008). Em outras palavras, utiliza-se o histórico de desempenho de algoritmos em uma coleção de bases de dados para treinar um *metamodelo* (modelo treinado sobre metadados de execuções anteriores, e não diretamente sobre os dados do problema-alvo) que pode prever o melhor algoritmo ou configuração para um novo problema. Adicionalmente, Schoenfeld, Seppi e Giraud-Carrier (2022) mostraram que, em vez de tratar modelos como caixas-pretas, pode-se explorar metadados granulares dos pipelines para melhorar a previsão de desempenho.

As abordagens de meta-aprendizado podem ser *offline* ou *online*. Em métodos *offline*, diversos pipelines são executados previamente em coleções de conjunto de dados e os resultados formam o meta conjunto usado para treinar o metamodelo. Sistemas AutoML como o Auto-sklearn Feurer *et al.* (2015) e o SmartML Maher e Sakr (2019) ilustram esse caso, utilizando os resultados de múltiplas execuções em bases conhecidas para guiar a busca em novas bases. Em contrapartida, métodos *online*, como o TPOT (OLSON *et al.*, 2016), atualizam o aprendizado à medida que geram pipelines no próprio problema de interesse.

Além disso, benchmarks específicos como o *meta conjunto*, proposto por Triantafillou *et al.* (2020), foram desenvolvidos para avaliar modelos de few-shot learning e meta-aprendizado em cenários mais realistas e desafiadores. O *meta conjunto* é composto por um conjunto diversificado de conjuntos de dados que permite avaliar a capacidade dos meta-modelos de generalizar para novos domínios e tarefas com poucos exemplos. A proposta introduz variações no número de classes, tamanho de conjuntos de suporte e testes com desbalanceamento, promovendo uma avaliação mais robusta dos métodos de meta-aprendizado.

Em todos os casos, o objetivo é transferir conhecimento entre tarefas: utilizar metadados e meta características coletados em problemas anteriores para reduzir o tempo e esforço de modelagem em novos problemas.

2.3 Meta-características e Caracterização de Conjuntos de Dados

A caracterização de dados por meio de *meta características* é essencial para o meta-aprendizado. Meta características são atributos que descrevem propriedades estatísticas e estruturais de um conjunto de dados, e servem de entrada para os metamodelos. Por exemplo, Schoenfeld, Seppi e Giraud-Carrier (2022) extraíram 72 meta características de cada base de dados, incluindo categorias simples, estatísticas, informações teóricas, *landmarking* e baseadas em modelo. Esses grupos podem ser definidos genericamente da seguinte forma:

- **Simples:** propriedades elementares do conjunto, como número de instâncias, número de atributos, proporção de classes;
- **Estatísticas:** medidas como médias, variâncias e correlações dos atributos;
- **Informacionais:** métricas de teoria da informação, como entropia das classes e entropia dos atributos;
- **Landmarking:** desempenho de algoritmos simples e rápidos (por exemplo, taxa de erro de 1-vizinho mais próximo, árvore de decisão rasa) no conjunto de dados;
- **Baseadas em modelo:** características derivadas de modelos treinados, como profundidade média de árvores de decisão, coeficientes de regressão, etc.

Essas meta características capturam informações relevantes que permitem medir similaridades entre conjuntos de dados e orientar o aprendizado Feurer, Springenberg e Hutter (2015). Muitas pesquisas utilizam repositórios de dados públicos; por exemplo, o conjunto de 194 bases usado por Schoenfeld, Seppi e Giraud-Carrier (2022) provém do programa D3M (*Data-Driven Discovery of Models*), iniciativa da DARPA voltada à automação da ciência de dados Lippmann, Campbell e Campbell (2016), e inclui conjuntos de dados originados de repositórios como o OpenML Vanschoren *et al.* (2014) e o UCI Machine Learning Repository (DUA; GRAFF, 2019). Cada instância do meta conjunto consiste no vetor de metacaracterísticas do conjunto de dados e na estrutura do pipeline avaliado, associado ao respectivo desempenho (por exemplo, medida F1) obtido naquela base. Esse meta conjunto (meta características + descrição de pipeline + desempenho) é então utilizado para treinar metamodelos capazes de prever ou ranquear pipelines em novos conjuntos de dados.

2.4 Aprendizado Ativo

O aprendizado ativo é uma abordagem em que o algoritmo seleciona interativamente quais instâncias devem ser rotuladas para melhorar seu desempenho com poucos dados rotulados Cohn, Atlas e Ladner (1994). Settles (2009) descreve que a ideia central do aprendizado

ativo é permitir que o sistema escolha os exemplos a serem rotulados, de forma que se atinja maior precisão com um número reduzido de etiquetas. Em outras palavras, um modelo com capacidade de consulta pode alcançar desempenho elevado utilizando apenas os exemplos mais informativos. Esse paradigma é motivado por cenários em que há abundância de dados não rotulados e o custo de rotular é alto.

Para implementar o aprendizado ativo, definem-se estratégias de consulta que estimam a utilidade de rotular cada instância disponível. Abordagens comuns incluem amostragem por incerteza (rotular as instâncias nas quais o modelo está menos confiante), seleção por comitê (vários modelos votam nas instâncias mais controversas) e redução de erro esperado.

No contexto de meta-aprendizado para AutoML, o aprendizado ativo é relevante como extensão futura: ao invés de avaliar todos os 60 pipelines em cada novo conjunto de dados, um sistema poderia selecionar ativamente quais pipelines avaliar primeiro com base na incerteza do metamodelo, reduzindo o custo computacional da construção do meta-conjunto. Doucet *et al.* (2025) e Sener e Savarese (2018) exemplificam essa abordagem ao utilizar *embeddings* para guiar a seleção de exemplos informativos no espaço de características — princípio análogo ao uso de *embeddings* de conjuntos de dados para identificar quais avaliações de pipeline são mais informativas para o sistema de recomendação.

2.5 Seleção e estrutura de *pipelines*

Em sistemas AutoML, um *pipeline* de aprendizado de máquina consiste na composição de múltiplos passos de pré-processamento e modelagem que transformam os dados até a predição final. Em geral, pipelines são representados como grafos acíclicos direcionados (DAGs), podendo incluir estruturas paralelas como comitês. Essa flexibilidade permite representar desde um pipeline simples (uma sequência linear de transformação e classificador) até configurações complexas (ex.: três sub pipelines com combinação por votação majoritária).

Entretanto, o número de pipelines possíveis cresce rapidamente com o número de algoritmos e passos disponíveis. Como observado por Schoenfeld, Seppi e Giraud-Carrier (2022), as diversas maneiras de combinar algoritmos em pipelines criam um espaço de busca *não escalonável* de possíveis fluxos a serem considerados. Diante disso, é útil empregar estratégias de pré-seleção ou ranqueamento de pipelines candidatos para priorizar a execução daqueles com maior potencial (especialmente quando recursos computacionais são limitados). Por exemplo, abordagens tradicionais de AutoML, como Auto-sklearn (FEURER *et al.*, 2015) e SmartML (MAHER; SAKR, 2019), tratam o pipeline como uma caixa-preta e utilizam apenas o desempenho observado e a similaridade de meta características do conjunto de dados para ranquear

modelos. Outras soluções simplificam a estrutura do pipeline, por exemplo considerando apenas sequências lineares e usando uma rede LSTM para estimar valor[1].

Em contraste, trabalhos mais recentes exploram explicitamente a estrutura do pipeline. Schoenfeld (2020) propõe um metamodelo baseado em rede neural dinamicamente montada, capaz de modelar DAGs arbitrários do pipeline. Schoenfeld, Seppi e Giraud-Carrier (2022) validam empiricamente essa abordagem criando um grande meta conjunto (cerca de 400.000 instâncias de aproximadamente 4.000 pipelines avaliados em 194 bases) e mostram que metamodelos que consideram metadados granulares de cada pipeline superam aqueles que usam representações simplificadas. Concretamente, eles demonstraram que metamodelos que utilizam toda a estrutura do pipeline fornecem estimativas de desempenho mais precisas, e que ranquear pipelines baseado na similaridade das meta características do conjunto de dados resulta em recomendações superiores às baseadas em estimativas de desempenho absolutas.

Em resumo, a seleção de pipelines em AutoML combina conhecimento do domínio (via meta-características de dados) e exploração da estrutura dos fluxos de processamento. Esse campo de pesquisa continua em desenvolvimento, mas já existe consenso de que considerar explicitamente a granularidade e a estrutura dos pipelines é crucial para sistemas AutoML eficientes e escaláveis.

2.6 Componentes do Arsenal Experimental

Como a metodologia deste trabalho avalia combinações de escalonamento, seleção de atributos e classificação, é necessário explicitar o papel dos operadores que compõem o arsenal experimental. Todos os componentes foram implementados por meio da biblioteca *scikit-learn*, referência consolidada para experimentação reprodutível em aprendizado de máquina com Python (PEDREGOSA *et al.*, 2011).

No estágio de escalonamento, foram considerados três transformadores e uma alternativa sem transformação. O `StandardScaler` centraliza cada atributo pela média e o reescala pelo desvio padrão, tornando os atributos comparáveis em termos de variância. O `MinMaxScaler` reescala os valores para um intervalo fixo, usualmente $[0,1]$. O `RobustScaler` utiliza mediana e intervalo interquartil, sendo menos sensível a valores extremos. Esses operadores são relevantes porque algoritmos baseados em distância, margem ou otimização numérica, como k-NN, SVM e MLP, podem ser fortemente afetados por diferenças de escala entre atributos.

No estágio de seleção ou redução de atributos, foram utilizados PCA, `SelectKBest` e ausência de seleção. A PCA projeta os dados em componentes ortogonais que concentram a maior variância possível, sendo uma técnica clássica de redução de dimensionalidade (JOL-

[1] LSTM: rede neural recorrente que aprende dependências de longo prazo em sequências por meio de um mecanismo de portas que regula o fluxo de informação em uma célula de memória interna Hochreiter e Schmidhuber (1997).

LIFFE, 2002). O `SelectKBest`, neste trabalho, utiliza o critério `f_classif`, baseado no teste F da ANOVA, para manter os atributos mais associados ao alvo; trata-se de uma estratégia de filtragem supervisionada de atributos (GUYON; ELISSEEFF, 2003). A alternativa `None` preserva todos os atributos originais, permitindo avaliar quando a redução dimensional prejudica ou favorece o desempenho.

No estágio de classificação, foram avaliados cinco algoritmos. O RF combina múltiplas árvores de decisão treinadas sobre amostras e subconjuntos de atributos, reduzindo variância e aumentando robustez em dados tabulares (BREIMAN, 2001). A SVM busca hiperplanos de máxima margem e, com kernel RBF, modela fronteiras não lineares (CORTES; VAPNIK, 1995). O k-NN é um método baseado em instâncias que classifica uma nova amostra pela vizinhança mais próxima no espaço de atributos (COVER; HART, 1967). A MLP é uma rede neural feedforward capaz de modelar relações não lineares por camadas ocultas (BISHOP, 2006). Por fim, a DT particiona recursivamente o espaço de atributos por regras de decisão, oferecendo interpretabilidade e baixo custo computacional (QUINLAN, 1986).

2.7 Embeddings de Conjuntos de Dados e Métricas de Ranking

Esta seção apresenta os dois fundamentos diretamente empregados na proposta deste trabalho: as representações vetoriais de conjuntos de dados conhecidas como *embeddings*, utilizadas para caracterizar os conjuntos de dados do meta-conjunto, e as métricas empregadas para avaliar a qualidade de um ranqueamento de pipelines.

2.7.1 Embeddings de Conjuntos de Dados

Uma limitação das meta-características tradicionais é que cada conjunto de dados é representado por um único vetor de estatísticas globais — médias, variâncias, entropia das classes —, o que descarta a variabilidade interna da distribuição dos atributos. Para superar essa limitação, abordagens mais recentes propõem *embeddings* de conjuntos de dados: vetores densos de dimensão fixa que capturam propriedades representativas do conjunto de dados de forma invariante ao número de atributos e instâncias.

Jomaa, Schmidt-Thieme e Grabocka (2021) propõem o Dataset2Vec, um *embedding* neural baseado em arquitetura DeepSet que representa o conjunto de dados como um conjunto hierárquico de pares preditores-alvo. O modelo é treinado com uma tarefa auxiliar de similaridade entre conjuntos de dados e produz vetores de 64 dimensões cujas proximidades no espaço euclidiano correlacionam-se com a similaridade de desempenho de algoritmos. Os autores identificam quatro critérios desejáveis para embeddings de conjuntos de dados, a saber: (D1) independência de esquema — aplicável a qualquer conjunto sem restrição de dimensionalidade; (D2) expressividade — capacidade de distinguir conjuntos com características de

aprendizado distintas; (D3) escalabilidade — calculável sem treinamento para novos conjuntos; (D4) correlação com meta-alvos — os vetores devem predizer desempenho de algoritmos.

Uma alternativa sem necessidade de treinamento consiste nos *embeddings* estatísticos distribucionais: ao invés de uma única estatística global por atributo, calcula-se a distribuição desta estatística sobre todos os atributos do conjunto de dados via percentis, produzindo um vetor de dimensão fixa que satisfaz os critérios D1 e D3 e se aproxima de D2 em tarefas tabulares. Esta abordagem é análoga ao MF2[2] de Feurer, Springenberg e Hutter (2015), porém com maior riqueza representacional por capturar distribuições e não apenas médias globais.

2.7.2 Métricas de Avaliação de Ranking

Na tarefa de ranqueamento de pipelines, a qualidade das predições é medida por métricas que comparam a ordenação predita com a ordenação real. As principais métricas utilizadas na literatura de meta-aprendizado são:

Coeficiente de Correlação de Spearman (ρ): mede a correlação monotônica entre os postos atribuídos aos scores preditos e reais. Valores próximos de 1 indicam que o ranking predito preserva a ordem correta dos pipelines; valores próximos de zero indicam ausência de correlação Spearman (1904).

Kendall Tau (τ): mede a fração líquida de pares de pipelines corretamente ordenados. Dado um par (i, j), o par é concordante se o ranking predito e o real concordam sobre qual pipeline é superior; é discordante caso contrário. O valor de τ é a diferença entre pares concordantes e discordantes, normalizada com tratamento de empates (tau-b na implementação) Kendall (1938).

Kendall Tau Ponderado (τ_w): extensão do Kendall Tau que atribui pesos maiores às inversões de ordem que ocorrem no topo do ranking. Implementado em `scipy.stats.weightedtau`, a configuração padrão usa pesos hiperbólicos aditivos $1/(r_i + 1) + 1/(r_j + 1)$ e média de duas ordenações lexicográficas decrescentes dos escores real e predito. Postos menores têm mais peso; a implementação não fornece p-valor.

NDCG (*Normalized Discounted Cumulative Gain*): métrica proveniente da área de recuperação de informação, que pondera a relevância dos itens recuperados pelo logaritmo da posição na lista. Em meta-aprendizado, o NDCG penaliza explicitamente pipelines de alto desempenho recomendados em posições distantes do topo.

A adoção conjunta dessas quatro métricas permite avaliar o sistema tanto em termos de correlação global (Spearman e Kendall Tau) quanto de precisão nas posições de maior impacto prático. Nesse segundo grupo, Kendall Tau ponderado e NDCG são métricas complementa-

[2] O MF2 é um conjunto de 46 meta-características estáticas — incluindo medidas de *landmarking*, componentes principais (PCA) e proporção de valores ausentes — originalmente proposto por Reif, Shafait e Dengel (2012) e empregado por Feurer, Springenberg e Hutter (2015) para estimar a similaridade entre conjuntos de dados na inicialização de otimização bayesiana de hiperparâmetros.

res: a primeira penaliza inversões pareadas no topo do ranking, enquanto a segunda mede a relevância acumulada das primeiras posições.

2.8 Trabalhos Relacionados

Esta seção apresenta os principais trabalhos relacionados à proposta desta pesquisa, organizados em três eixos temáticos: (i) abordagens para extração de meta-características e representação de conjuntos de dados; (ii) sistemas de recomendação de algoritmos e pipelines de aprendizado de máquina via meta-aprendizado; e (iii) estratégias de avaliação de complexidade de fronteiras de decisão e aprendizado ativo. Ao final, é apresentada uma síntese comparativa que posiciona a presente proposta em relação ao estado da arte.

2.8.1 Representação de Conjuntos de Dados e Meta-características

A qualidade dos metamodelos em sistemas de meta-aprendizado depende diretamente da capacidade de representar conjuntos de dados de forma compacta e informativa. Historicamente, duas correntes principais se desenvolveram: a engenharia de meta-características estáticas e a aprendizagem de meta-características por redes neurais.

Vanschoren *et al.* (2014) introduzem o OpenML, uma plataforma colaborativa que centraliza conjuntos de dados, experimentos e resultados de algoritmos em um repositório público e estruturado. Além de prover acesso unificado a centenas de conjuntos de dados anotados, o OpenML define uma infraestrutura para coleta sistemática de meta-dados — acurácias, tempos de execução, hiperparâmetros utilizados — que viabiliza a construção de grandes meta-conjuntos. Esta plataforma constitui a principal fonte de dados do presente trabalho, tendo fornecido os 201 conjuntos utilizados nos experimentos.

Schoenfeld, Seppi e Giraud-Carrier (2022) investigam a construção de meta-conjuntos de grande escala para otimização de pipelines, criando uma base com aproximadamente 400 mil instâncias derivadas de 4 mil pipelines avaliados em 194 conjuntos do programa D3M. O trabalho demonstra empiricamente que metamodelos que consideram metadados granulares de toda a estrutura do pipeline superam abordagens que utilizam apenas representações simplificadas. Os autores mostram, ainda, que o ranqueamento baseado em similaridade de meta-características fornece recomendações superiores às estimativas de desempenho absoluto, convergindo diretamente com a abordagem adotada neste TCC.

Jomaa, Schmidt-Thieme e Grabocka (2021) propõem o *Dataset2Vec*, um extrator de meta-features baseado em uma arquitetura DeepSet que representa conjuntos de dados como conjuntos hierárquicos de pares preditores-alvo. O modelo é treinado com uma tarefa auxiliar de aprendizagem de similaridade entre conjuntos de dados (*dataset similarity learning*), que determina se dois subconjuntos amostrados provêm do mesmo conjunto. Os resultados expe-

rimentais com 120 conjuntos UCI para a tarefa de otimização de hiperparâmetros demonstram que as meta-features aprendidas pelo Dataset2Vec superam meta-features estáticas tradicionais (MF1 de Wistuba, Schilling e Schmidt-Thieme (2016) com 22 dimensões; MF2 de Feurer, Springenberg e Hutter (2015) com 46 dimensões) e o *Neural Statistician* (Edwards e Storkey (2017)) em termos de correlação de Spearman com os alvos de desempenho.

O presente trabalho diferencia-se do Dataset2Vec em dois aspectos fundamentais: (i) em lugar de embeddings neurais, utiliza-se um embedding distribucional de 64 dimensões, produzido sem necessidade de treinamento prévio, com propriedades análogas ao MF2 mas com maior riqueza por capturar a *distribuição* das meta-features por atributo (não apenas médias globais); (ii) a tarefa meta-nível é o *ranqueamento de pipelines* (não apenas otimização de hiperparâmetros), sobre um meta-conjunto de 201 conjuntos de dados. Esta escolha de embedding estático garante a propriedade de independência de esquema (**D1** na taxonomia do Dataset2Vec), escalabilidade sem treinamento (**D3**) e reprodutibilidade, ao custo de expressividade potencialmente menor (**D2**) em relação a abordagens neurais.

2.8.2 Sistemas de Recomendação de Algoritmos e Pipelines

Schoenfeld (2020) propõem um metamodelo baseado em rede neural dinamicamente configurável, capaz de modelar DAGs arbitrários de pipelines. O sistema representa cada operador do pipeline com uma subrrede específica e combina as representações por uma função de pooling global. Esta abordagem permite estimar o desempenho de configurações de pipeline inéditas sem avaliação direta, tornando a exploração do espaço de busca mais eficiente. A arquitetura supera metamodelos com representações simplificadas de pipeline, reforçando a importância de considerar a estrutura interna dos fluxos de processamento.

Golshanrad *et al.* (2021) apresentam o sistema MEGA (*Meta-learning Ensemble for Genetic Algorithm*), um método de combinação de classificadores que emprega meta-aprendizado e algoritmo genético para selecionar a melhor combinação de classificadores para um conjunto de dados desconhecido. O sistema extrai meta-características dos conjuntos, usa um algoritmo genético para encontrar combinações eficazes e, na fase de inferência, aplica um k-NN ponderado para recomendar combinações ideais. O uso do k-NN como metamodelo de inferência é compatível com a abordagem adotada neste trabalho, onde também se emprega k-NN para predição de rankings de pipelines.

Pereira, Santos e Carvalho (2021) investigam a recomendação de algoritmos de seleção de atributos por meta-aprendizado, modelando a tarefa como um problema supervisionado. O trabalho emprega k-NN, SVM, árvores de decisão e florestas aleatórias como meta-aprendizes, com meta-características extraídas de 28 meta-conjuntos do repositório ASlib. Os autores demonstram que técnicas de redução de dimensionalidade (PCA, seleção por variância, Chi-quadrado, ANOVA-F) permitem reduzir em mais de 80% o número de meta-características

sem degradar o desempenho preditivo, evidenciando a robustez de abordagens baseadas em similaridade para recomendação.

Wang, Song e Zhu (2021) propõem um método multi-rótulo de recomendação de algoritmos de classificação baseado em aprendizado por comitê. O modelo mapeia meta-características de 1090 problemas de classificação para o conjunto de algoritmos adequados, combinando múltiplos tipos de meta-características em modelos-base com integração ponderada. Os experimentos demonstram superioridade em métricas de ranking (perda de ordenação e *Precision@m*) sobre modelos individuais, reforçando o valor de abordagens de comitê para recomendação.

Khan *et al.* (2023) descrevem o AutoFE-Sel, sistema de meta-aprendizado para recomendação de algoritmos de seleção de subconjunto de atributos. A abordagem utiliza a métrica EARR (*Adjusted Ratio of Ratios*), que combina acurácia, tempo de execução e número de atributos selecionados para comparar algoritmos. Experimentos com 125 conjuntos de dados e 12 algoritmos candidatos demonstram que o AutoFE-Sel supera abordagens tradicionais (MCFA, IBL-Ranking, Fuzzy-Sim) em acurácia, *hit-ratio* e *F-measure*, atingindo acerto superior a 92%.

2.8.3 Avaliação de Fronteiras de Decisão e Aprendizado Ativo

Varshney e Ramamurthy (2015) utilizam topologia persistente para caracterizar a complexidade das fronteiras de decisão em classificadores supervisionados, construindo complexos simpliciais a partir de amostras com rótulos distintos. Os números de Betti obtidos pela homologia persistente correlacionam-se com o desempenho preditivo dos modelos de SVM, auxiliando na seleção de hiperparâmetros. Embora a abordagem topológica seja distinta da proposta neste TCC, o princípio de usar características intrínsecas dos dados para orientar a seleção de modelos é compartilhado.

Ramamurthy, Varshney e Mody (2018) expandem a análise topológica propondo os complexos *labeled Čech* e *labeled Vietoris-Rips* para estimar a complexidade topológica da fronteira de decisão (DBTC). Experimentos em MNIST, FashionMNIST e CIFAR-10 mostram que a correspondência entre a DBTC do modelo e a do conjunto de dados favorece a seleção de modelos pré-treinados, evidenciando que propriedades estruturais dos dados podem guiar a escolha de algoritmos.

Peng (2020) apresentam uma revisão abrangente sobre meta-aprendizado, sistematizando métodos baseados em otimização, modelo, métrica e inferência Bayesiana. O trabalho discute como o aprendizado meta-episódico permite generalização eficiente para novas tarefas com dados rotulados limitados, especialmente em cenários de *few-shot learning* (aprendizado a partir de pouquíssimos exemplos rotulados por classe), formalizando a notação de meta-tarefas $\tau_i \sim \mathcal{P}(\mathcal{T})$ com conjuntos de suporte $\mathcal{D}_i^s$ e consulta $\mathcal{D}_i^q$.

Doucet *et al.* (2025) propõem o método TCM, que combina amostragem baseada em diversidade e incerteza para aprendizado ativo. O TCM realiza uma transição gradual de *TypiClust*

— que seleciona exemplos representativos por clustering em embeddings auto-supervisionados — para *Margin Sampling*, focado na incerteza do modelo. Os autores demonstram superioridade sobre BADGE, Coreset e ProbCover em CIFAR-10, CIFAR-100 e ISIC2019, evidenciando o valor de combinar diversidade e incerteza na seleção de instâncias informativas. A utilização de embeddings para identificar exemplos representativos é análoga ao uso de embeddings de conjuntos de dados neste trabalho para identificar conjuntos similares no espaço de meta-aprendizado.

2.8.4 Síntese Comparativa

A Tabela 2 sintetiza os principais trabalhos relacionados em relação a quatro dimensões: tipo de representação dos dados, tarefa de meta-nível, tamanho do meta-conjunto e método de avaliação utilizado.

Tabela 2 – Comparativo entre trabalhos relacionados e a proposta deste TCC

Trabalho	Representação	Tarefa Meta-nível	Meta-conjunto	Avaliação
Schoenfeld et al. (2022)	Estrutura do pipeline (DAG)	Estimativa de desempenho	∼400 k instâncias	Correlação de Spearman
Dataset2Vec (2021)	Embedding neural (DeepSet)	Otimização de hiperparâmetros	120 conjuntos UCI	Spearman (ranqueamento)
MEGA (2020)	Meta-características estáticas	Recomendação de comitês	Múltiplos conjuntos	Acurácia
AutoFE-Sel (2021)	Meta-características estáticas	Seleção de atributos	125 conjuntos	EARR, hit-ratio
Wang et al. (2021)	Meta-características multi-tipo	Recomendação multi-rótulo	1090 conjuntos	Ranking loss, Prec@m
Este TCC	**embedding distribucional 64D**	**Ranqueamento de pipelines**	**193 tarefas / 165 grupos / 60 pipelines**	**Spearman, Kendall (τ, τ_w), NDCG**

Fonte: Autoria própria (2026).

A análise comparativa revela que a presente proposta distingue-se dos trabalhos anteriores principalmente por três aspectos: (i) foco explícito no *ranqueamento completo de pipelines* (não apenas seleção do melhor ou estimativa pontual); (ii) adoção conjunta de **Kendall Tau ponderado** (*weightedtau*) e NDCG como métricas complementares de topo, alinhando a avaliação às necessidades reais de AutoML, onde os primeiros pipelines recomendados têm maior impacto prático; e (iii) análise estatística da correlação entre dimensões do embedding e qualidade de ranqueamento por conjunto de dados, investigando *quando* e *por que* o meta-aprendizado funciona melhor.

A escolha de uma representação estatística distribucional, referida ao longo do trabalho como embedding distribucional por produzir um vetor denso de dimensão fixa, foi motivada por dificuldades de compatibilidade no ambiente original e pelo objetivo de estabelecer uma referência determinística e auditável. A escolha não depende de uma afirmação permanente sobre a disponibilidade de versões do TensorFlow. Diferentemente de embeddings neurais como o Dataset2Vec, essa representação não é aprendida por otimização de uma função objetivo; ela se aproxima do MF2 de Feurer et al. (2015) em termos de engenharia de meta-características, mas captura distribuições de meta-features por atributo em vez de apenas estatísticas globais do conjunto de dados.

2.9 Avaliação Contemporânea de AutoML e Meta-aprendizado

Três eixos complementares orientam a atualização: representação, recomendação e avaliação. No primeiro, Hollmann *et al.* (2025) apresentam TabPFN, um modelo fundacional tabular que exemplifica reutilização de conhecimento entre problemas. Sua tarefa de predição difere do ranqueamento de um arsenal fixo; seus resultados não são comparação numérica direta.

No eixo de recomendação, Baratchi *et al.* (2024) distinguem espaço de busca, estratégia e avaliação. Esse enquadramento separa representação, metamodelo e candidatos disponíveis. Um arsenal fixo facilita comparação controlada, mas limita conclusões sobre espaços de hiperparâmetros mais amplos.

No eixo de avaliação, Gijsbers *et al.* (2024) apresentam AMLB, um benchmark extensível que considera desempenho, custo e falhas. Essa perspectiva motiva explicitar resultados ausentes e protocolos. Sua aplicação aqui é uma decisão metodológica, não um resultado transferido do AMLB.

Goldblum *et al.* (2024) discutem os pressupostos dos teoremas No Free Lunch e os vieses de problemas reais. O teorema clássico não demonstra que todos os métodos tenham desempenho equivalente em benchmarks práticos, nem garante ganho de um recomendador específico. O ganho precisa ser medido no domínio de interesse.

3 MATERIAIS E MÉTODOS

Este capítulo descreve os materiais, ferramentas computacionais, conjuntos de dados e métodos empregados no desenvolvimento desta pesquisa. Inicialmente são apresentados os recursos utilizados, seguidos de um panorama geral do processo experimental baseado em dois fluxogramas. Por fim, detalha-se cada etapa metodológica da Fase 1 e Fase 2 do experimento de meta-aprendizado para recomendação de pipelines de aprendizado de máquina.

3.1 Materiais Utilizados

Nesta seção, são apresentados os recursos computacionais, linguagens de programação, bibliotecas, ambientes de desenvolvimento e conjuntos de dados empregados ao longo da pesquisa. A seleção desses elementos foi guiada por critérios como aderência às práticas consolidadas na área de Aprendizado de Máquina (ML), disponibilidade em código aberto, escalabilidade e suporte à experimentação científica.

Antes de detalhar cada grupo de materiais individualmente, é oportuno oferecer uma visão geral do ecossistema tecnológico que fundamenta esta investigação. O Quadro 1 a seguir consolida os principais componentes utilizados, organizando-os por categoria, versão (quando aplicável) e justificativa de sua escolha. Esta apresentação permite ao leitor compreender, de forma estruturada, a base técnica sobre a qual os experimentos foram conduzidos.

Quadro 1 – Materiais, Ferramentas e Conjuntos de Dados Utilizados

Categoria	Nome Específico	Versão	Justificativa/Relevância
Linguagem	Python	3.13	Ecossistema robusto para AM, vastas bibliotecas
Ambiente de Desenvolvimento	Jupyter / VS Code	N/A	Experimentação interativa, gerenciamento de código
Biblioteca de AM	Scikit-learn	$\geq$ 1.4	Pipelines, pré-processamento, classificadores e métricas
Estatística Computacional	SciPy	$\geq$ 1.12	KDTree, correlações e métricas de ranqueamento
Meta-características	Implementação própria	N/A	Extração de embeddings estatísticos distribucionais
Manipulação de Dados	Pandas	$\geq$ 1.3	Estrutura *DataFrame* para dados tabulares
Computação Numérica	NumPy	$\geq$ 1.20	Arrays multidimensionais e operações matemáticas
Visualização	Matplotlib, Seaborn e umap-learn	N/A	Gráficos, mapas de calor e projeção UMAP
Fonte de Dados	OpenML	N/A	Repositório com conjuntos anotados e metadados
Recurso Computacional	PC local	Próprio	Execução dos experimentos das Fases 1 e 2

Fonte: Autoria própria (2026).

O quadro destaca a integração entre ferramentas consolidadas do ecossistema científico de Python e infraestrutura computacional local, evidenciando o cuidado na seleção de tecnologias que garantam reprodutibilidade, desempenho e flexibilidade. A seguir, cada categoria é explorada individualmente, de forma a detalhar seu papel específico no desenvolvimento dos experimentos desta pesquisa.

Os sites oficiais das principais ferramentas utilizadas são: Python[1], Jupyter[2], Visual Studio Code[3], scikit-learn[4], SciPy[5], NumPy[6], pandas[7], umap-learn[8] e OpenML[9].

3.1.1 Linguagens e Ambientes de Desenvolvimento

O projeto foi desenvolvido em Python (versão 3.13), utilizando como ambiente principal o Jupyter Notebook e, em etapas mais estruturadas, o Visual Studio Code. Essas ferramentas foram escolhidas devido à sua ampla adoção na comunidade científica e à compatibilidade com bibliotecas de aprendizado de máquina. O uso de controle de versão via `Git` e `GitHub` também foi empregado para rastreamento do código.

3.1.2 Bibliotecas e Frameworks

As principais bibliotecas utilizadas foram:

- `scikit-learn`: Para construção de pipelines, pré-processamento, validação cruzada e algoritmos base (RF, SVM, k-NN, MLP e DT) (PEDREGOSA *et al.*, 2011);
- `scipy`: Para a estrutura `KDTree`, correlações de Spearman e Kendall, e cálculo do Kendall Tau ponderado;
- `pandas` e `numpy`: Para manipulação e estruturação de dados tabulares e vetoriais;
- `matplotlib`, `seaborn` e `umap-learn`: Para geração dos gráficos, mapas de calor e projeção bidimensional dos embeddings.

3.1.3 Conjuntos de Dados

Os conjuntos de dados foram obtidos do repositório público OpenML (VANSCHOREN *et al.*, 2014), selecionando-se tarefas de classificação supervisionada que atendessem aos seguintes critérios: (i) entre 500 e 10.000 instâncias, de modo a equilibrar representatividade estatística e custo computacional; (ii) ao menos duas classes; e (iii) disponibilidade para processamento

[1] https://www.python.org/
[2] https://jupyter.org/
[3] https://code.visualstudio.com/
[4] https://scikit-learn.org/
[5] https://scipy.org/
[6] https://numpy.org/
[7] https://pandas.pydata.org/
[8] https://umap-learn.readthedocs.io/
[9] https://www.openml.org/

tabular. O carregador histórico codifica atributos categóricos; não há filtro implementado exigindo maioria numérica. Dos 243 *task IDs* configurados inicialmente, **201 conjuntos de dados** foram carregados com sucesso e tiveram ao menos um pipeline avaliado sem erros — número que supera os 120 conjuntos do UCI Repository utilizados por (JOMAA; SCHMIDT-THIEME; GRABOCKA, 2021) como referência na literatura para tarefas de meta-aprendizado. As 42 tarefas restantes não integram as matrizes; os artefatos não permitem reconstruir individualmente todos os motivos. Na revisão, critérios de qualidade são auditados por tarefa.

A caracterização local dos 201 conjuntos carregados indica heterogeneidade suficiente para avaliar o ranqueamento de pipelines: o número de instâncias varia de 500 a 10.000, com mediana de 1.941; o número de atributos varia de 1 a 1.776, com mediana de 20; e a mediana da proporção da classe majoritária é 0,50. Apenas um conjunto apresentou valores ausentes após o carregamento, caso tratado pela rotina de pré-processamento. A Tabela 3 apresenta exemplos representativos do meta-conjunto.

Tabela 3 – Exemplos de conjuntos de dados utilizados no meta-conjunto

Task ID	Nome	Caracterização	Instâncias	Atributos	Classes	Classe majoritária
3	`kr-vs-kp`	Jogo de xadrez	3.196	36	2	0,522
11	`balance-scale`	Balanceamento físico	625	4	3	0,461
9910	`Bioresponse`	Classificação bioquímica	3.751	1.776	2	0,542
363713	`regensburg_pediatric_appendicitis`	Diagnóstico médico tabular	780	55	3	0,619

Fonte: Autoria própria (2026), com base nos dados carregados do OpenML..

3.1.4 Recursos Computacionais

Os experimentos foram executados integralmente em hardware local, em uma estação de trabalho pessoal com as seguintes especificações:

- Processador: AMD Ryzen 7 5700U com Gráficos Radeon (8 núcleos, 16 threads, 1,80 GHz base);
- Memória RAM: 12 GB DDR4;
- Armazenamento: SSD (Solid-State Drive), utilizado como cache dos conjuntos de dados e dos embeddings, garantindo operações de leitura e escrita de alta velocidade durante a manipulação dos dados;
- Sistema Operacional: Windows 10 Pro (*build* 19045);
- Ambiente: Python 3.13, com as bibliotecas `scikit-learn` ($\geq$ 1.4), `scipy` ($\geq$ 1.12), `numpy`, `pandas` e `umap-learn`.

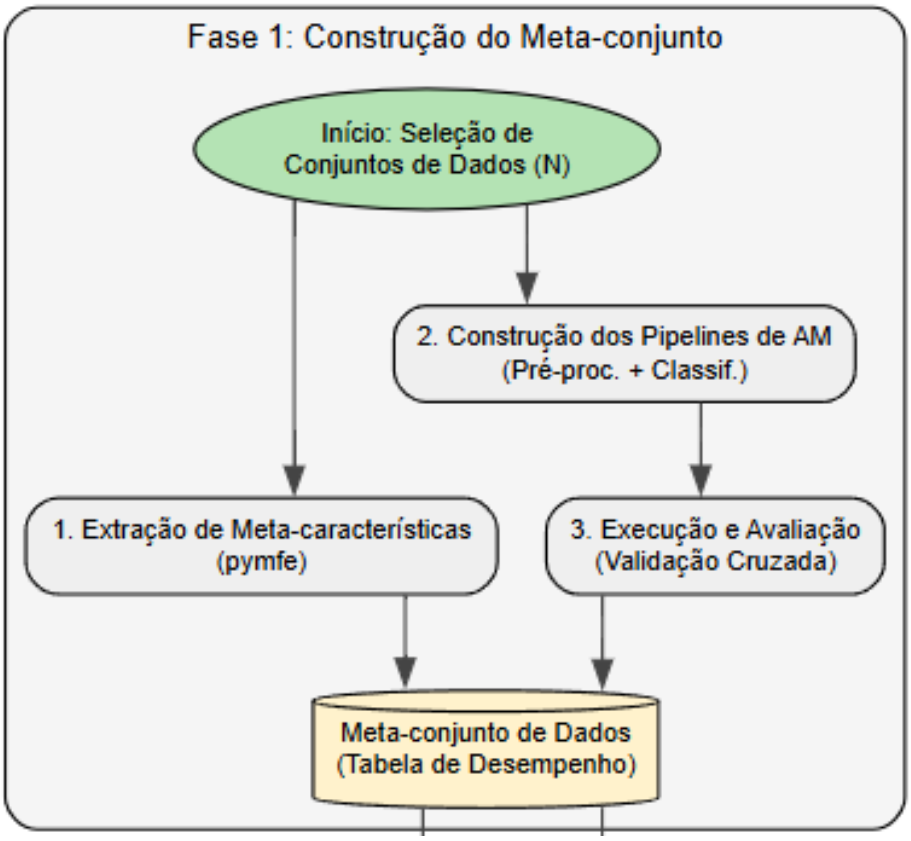
Esta configuração mostrou-se adequada para a execução dos experimentos: a Fase 1 (execução de 60 pipelines $\times$ 201 conjuntos de dados), intensiva em CPU e I/O, foi paraleli-

zada sobre os núcleos do processador; e a Fase 2 (validação LOO do preditor k-NN), de custo computacional linear no número de conjuntos de dados, foi concluída em poucos minutos.

3.2 Métodos

O processo experimental foi dividido em duas fases principais. A Fase 1 consiste na construção de um meta-conjunto de dados, onde pipelines são aplicados a conjuntos de dados previamente caracterizados. A Fase 2 realiza o meta-aprendizado e fornece a recomendação de pipelines adequados.

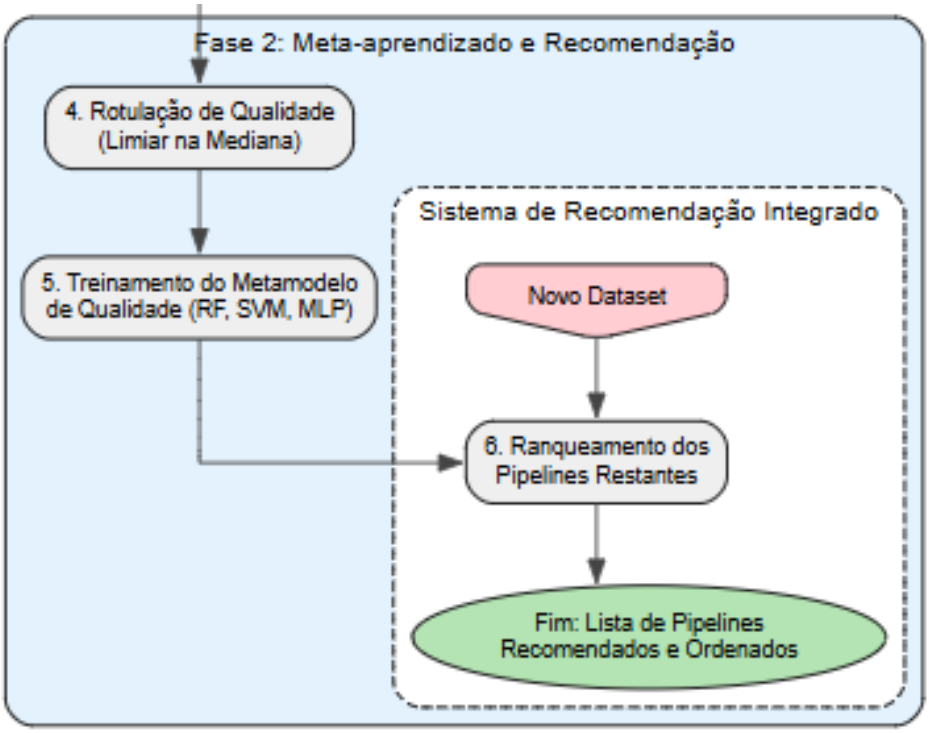
Figura 1 – Fluxograma do processo experimental: Fase 1 - Construção do meta-conjunto



Fonte: Autoria própria (2026).

O processo inicia com a seleção de conjuntos de dados, seguida da extração do embedding estatístico distribucional, da execução de pipelines (pré-processamento + classificador) e da avaliação via validação cruzada. Os dados resultantes são organizados no meta-conjunto com informações como: embedding do conjunto de dados (X_{emb}), configuração do pipeline e desempenho obtido (Y_{perf}).

Figura 2 – Fluxograma do processo experimental: Fase 2 - Meta-aprendizado e recomendação no processo experimental



Fonte: Autoria própria (2026).

Na Fase 2, o meta-conjunto é utilizado para treinar e avaliar um preditor de ranqueamento multi-saída. Para um novo conjunto de dados, o sistema localiza os conjuntos mais similares no espaço de embeddings e estima a acurácia esperada de cada pipeline, gerando uma lista ordenada de recomendações.

A seguir, cada fase do processo experimental é pormenorizada. A Fase 1 foca na criação da base de conhecimento – o meta-conjunto de dados –, que é a fundação para a abordagem de meta-aprendizado. A Fase 2 utiliza essa base para treinar os metamodelos e, finalmente, gerar as recomendações de pipelines, que constitui o objetivo central desta pesquisa.

3.2.1 Fase 1 – Construção do Meta-conjunto

A representação dos conjuntos rotulados é um vetor determinístico de meta-atributos estatísticos, também denominado embedding estatístico, sem treinamento, função de perda ou otimização da representação. Para cada atributo numérico são calculados nove descritores: média, desvio padrão populacional acrescido de 10^{-8}, mínimo, máximo, mediana, assimetria, curtose de Fisher, correlação de Pearson com o alvo e correlação de Spearman com o alvo. Cada descritor é agregado entre atributos por mínimo, percentil 25, média, percentil 75 e máximo, nesta ordem, produzindo $9 \times 5 = 45$ entradas. Os percentis por atributo calculados pelo código não integram os nove blocos.

O bloco global contém, nesta ordem: $\log(1+N)$, $\log(1+P)$, número de classes, proporção majoritária, proporção minoritária, entropia das proporções acrescidas de 10^{-8}, proporção de atributos constantes, média da correlação absoluta entre atributos com diagonal zerada, proporção média de valores únicos por atributo e $P/(N+10^{-8})$. A média de correlações inclui os zeros da diagonal no denominador P^2. Os blocos totalizam 55 descritores; com dimensão configurada em 64, acrescentam-se nove zeros, sem truncamento. Valores não finitos são tratados por `nan_to_num`, e o vetor é normalizado pela norma L2, exceto quando nulo.

O contrato de índices e operações consta em `docs/revisao/EMBEDDING.md`. A representação exige rótulos disponíveis do conjunto de entrada; correlações com rótulos multiclasses codificados dependem de sua ordenação nominal. A padronização entre dimensões, quando avaliada, é aprendida somente no treino do metamodelo. O caminho estatístico é distinto do Dataset2Vec neural (JOMAA; SCHMIDT-THIEME; GRABOCKA, 2021); as matrizes versionadas fixam qual representação foi efetivamente utilizada.

Os pipelines são compostos por sequências de operadores de pré-processamento seguidos por um classificador. A modularidade dos componentes permite gerar múltiplas combinações, cujas estruturas são registradas de forma detalhada para compor o histórico do experimento.

Cada pipeline é avaliado por meio de validação cruzada estratificada de 5 folds sobre os conjuntos de dados. A métrica de desempenho adotada é a **acurácia balanceada** (*balanced accuracy*), definida como a média aritmética da sensibilidade (recall) de cada classe, conforme a Equação (1) Brodersen *et al.* (2010):

$$\text{BA} = \frac{1}{C} \sum_{c=1}^{C} \frac{TP_c}{TP_c + FN_c} \tag{1}$$

onde C é o número de classes, TP_c é o número de verdadeiros positivos e FN_c o de falsos negativos para a classe c. Para $C = 2$, a acurácia balanceada equivale à média aritmética de sensibilidade e especificidade. A escolha desta métrica em detrimento da acurácia simples é motivada pela presença de conjuntos de dados desbalanceados no meta-conjunto: a acurácia simples favorece artificialmente classificadores que preveem apenas a classe majoritária, enquanto a acurácia balanceada garante que o desempenho em todas as classes seja igualmente considerado. O desempenho médio sobre os 5 folds compõe o valor Y_{perf} armazenado na matriz de acurácias.

Ao final da Fase 1, o principal artefato gerado é o meta-conjunto de dados, que consolida o conhecimento empírico obtido a partir da execução sistemática de diversos pipelines sobre uma ampla variedade de conjuntos de dados. Este meta-conjunto é estruturado de forma que cada instância representa um par formado por um conjunto de dados e uma configuração de pipeline, contendo as seguintes informações:

- **Embedding estatístico do conjunto de dados (X_{emb}):** representa as características intrínsecas do conjunto de dados por meio de estatísticas distribucionais, servindo como entrada para o preditor de ranqueamento;
- **Configuração do pipeline:** descreve a sequência de operadores de pré-processamento e o classificador utilizados, bem como seus respectivos hiperparâmetros;
- **Desempenho do pipeline (Y_{perf}):** armazena os resultados das métricas obtidas durante a validação cruzada, como acurácia balanceada e F1-score.

3.2.2 Fase 2 – Meta-aprendizado por Ranqueamento

Com o meta-conjunto construído, a Fase 2 realiza a **predição de ranking** de pipelines para novos conjuntos de dados. Esta abordagem trata a recomendação como um problema de ranqueamento (em vez de classificação binária), o que preserva toda a informação ordinal de desempenho e permite ao usuário selecionar quantos pipelines desejar da lista recomendada.

3.2.2.1 Preditor k-NN Multi-saída

O metamodelo adotado é um **preditor k-vizinhos mais próximos** (k-NN) multi-saída, que utiliza uma estrutura `KDTree`[10] para busca eficiente de vizinhos no espaço de embeddings. A KDTree é uma árvore binária de particionamento espacial que organiza os pontos por planos de corte em cada dimensão, permitindo consultas cuja eficiência depende da dimensão e da distribuição, sem garantia de custo logarítmico em alta dimensão; n é o número de conjuntos de dados e $d = 64$ é a dimensão do embedding. Para um novo conjunto de dados D_{new} com embedding $\mathbf{x}_{new} \in \mathbb{R}^{64}$, o preditor identifica os k conjuntos de dados mais similares no espaço de embeddings e estima a acurácia esperada de cada pipeline P_j como a média das acurácias observadas nos vizinhos, conforme a Equação (2) — generalização multi-saída da regressão k-NN clássica, em que a predição é dada pela média ponderada das observações dos k vizinhos mais próximos Altman (1992):

$$\hat{A}(P_j) = \frac{\sum_{D_i \in \mathcal{N}_k(D_{new})} w_i A_{i,j}}{\sum_{D_i \in \mathcal{N}_k(D_{new})} w_i}, \quad w_i = \frac{1}{\|\mathbf{x}_{new} - \mathbf{x}_i\|_2 + 10^{-8}} \tag{2}$$

onde $\mathcal{N}_k(D_{new})$ denota o conjunto dos k conjuntos de dados mais próximos de D_{new} no espaço de embeddings e $A_{i,j}$ é a acurácia balanceada do pipeline P_j no conjunto de dados D_i. A similaridade entre conjuntos de dados é medida pela distância euclidiana no espaço de em-

[10] A KDTree (*k-dimensional tree*) é uma estrutura de árvore binária que particiona recursivamente o espaço de busca por planos de corte alinhados aos eixos, permitindo consultas de vizinhança eficientes em espaços multidimensionais Bentley (1975).

beddings. A recomendação final consiste no ranking dos 60 pipelines ordenados por $\hat{A}(P_j)$ de forma decrescente.

3.2.2.2 Validação por Leave-One-Out

A avaliação do preditor é realizada por **Leave-One-Out (LOO) no nível de conjuntos de dados**: para cada conjunto D_i no meta-conjunto, o preditor é treinado nos $n-1$ conjuntos restantes e avalia suas predições sobre o conjunto excluído. Esse protocolo histórico exclui uma tarefa por vez, mas pode manter outra relacionada no treino; por isso, sua reprodução é exploratória e a avaliação principal da revisão separa grupos de tarefas relacionadas.

3.2.2.3 Métricas de Avaliação de Ranking

A qualidade do ranqueamento predito em relação ao ranqueamento real foi avaliada por quatro métricas complementares:

- **Coeficiente de Correlação de Spearman (ρ)** (SPEARMAN, 1904): mede a correlação monotônica entre os postos dos scores preditos e reais. $\rho = 1$ indica ordenação perfeita; $\rho = 0$, ausência de correlação linear de postos;
- **Kendall Tau (τ)** (KENDALL, 1938): mede a fração líquida de pares de pipelines corretamente ordenados, conforme a Equação (3):

$$\tau_b = \frac{P_c - Q_d}{\sqrt{(P_c + Q_d + T_x)(P_c + Q_d + T_y)}} \tag{3}$$

  onde P_c,Q_d contam pares concordantes e discordantes e T_x,T_y contam empates exclusivos em cada ranking. $\tau = 1$ indica todos os pares na ordem correta; $\tau = -1$, ordem invertida;

- **Kendall Tau Ponderado (τ_w)**: implementado via `scipy.stats.weightedtau`, atribui pesos maiores às inversões de ordem que ocorrem no topo do ranking (melhores pipelines), penalizando erros de recomendação que têm maior impacto prático. Os parâmetros padrão usam peso aditivo $1/(r_i + 1) + 1/(r_j + 1)$ e média de duas ordenações lexicográficas decrescentes dos escores real e predito, sem p-valor definido;
- **NDCG** (*Normalized Discounted Cumulative Gain*): implementado via `sklearn.metrics.ndcg_score`, penaliza explicitamente pipelines relevantes classificados em posições inferiores, definido pela Equação (4):

$$\text{NDCG} = \frac{\text{DCG}}{\text{IDCG}}, \quad \text{DCG} = \sum_{i=1}^{n} \frac{rel_i}{\log_2(i+1)} \tag{4}$$

onde rel_i é a relevância do pipeline na posição i (acurácia balanceada real) e IDCG é o DCG do ranking ideal (ordenação perfeita). O ganho é linear, conforme a implementação usada. Relevâncias próximas podem produzir NDCG alto mesmo com muitas inversões; a revisão acrescenta NDCG@5 e regret sem interpretar NDCG alto como recuperação exata do top-5. Assim, τ_w e NDCG são tratados como métricas complementares de topo: o primeiro avalia inversões pareadas ponderadas, enquanto o segundo avalia a relevância acumulada das posições iniciais.

As métricas são reportadas como média e desvio padrão sobre todos os folds LOO válidos (folds com variância não nula no vetor de acurácias reais). Para a seleção do hiperparâmetro k, avalia-se $k \in \{1, 3, 5, 7, 10\}$ como exploração histórica. Na avaliação revisada, k é escolhido pela média de Kendall ponderado por grupo em validação interna, sem consultar o teste externo.

3.2.2.4 Análise de Correlação Estatística

Como análise complementar, investiga-se a **correlação estatística entre o Kendall Tau ponderado por conjunto de dados e as dimensões do embedding**. Para cada dimensão $d \in \{1, \ldots, 64\}$ do espaço de embeddings, calcula-se o coeficiente de Spearman entre $\{\tau_w^{(i)}\}_{i=1}^{n}$ (scores de qualidade de ranqueamento por conjunto de dados) e $\{x_d^{(i)}\}_{i=1}^{n}$ (valor da dimensão d no embedding de cada conjunto). Esta análise visa identificar quais aspectos da representação do conjunto de dados predizem a qualidade do meta-aprendizado, fornecendo interpretabilidade ao sistema.

3.2.3 Auditoria e Validação por Grupos

Metadados oficiais identificam tipo de tarefa, conjunto e alvo. Nomes repetidos e fingerprints dos atributos processados formam grupos transitivos conservadores. Uma tarefa é elegível se é de classificação supervisionada, possui 500 a 10.000 exemplos, ao menos duas classes e cinco observações por classe. O catálogo registra critérios e exclusões sem usar a qualidade do ranqueamento. A tarefa 363713 não é holdout independente, pois integra o acervo.

A avaliação externa usa `GroupKFold` com cinco divisões, embaralhamento e semente 42. Em cada treino externo, quatro divisões internas por grupos selecionam $k \in \{1,3,5,7,10\}$ pela média de Kendall ponderado por grupo; empates escolhem o menor k. Grupos de teste ficam ausentes do treino, inclusive nas divisões internas. Há dois k-NN ajustados: um com os vetores L2 históricos e outro com StandardScaler ajustado no treino de cada divisão.

Random Forest é um regressor multi-saída nativo, com 200 árvores, mínimo de duas observações por folha, `max_features=1.0` e semente 42. As baselines são média por pi-

peline somente no treino e escores aleatórios uniformes com semente 42 acrescida do índice do fold. Todos os modelos usam as mesmas partições.

Saídas ausentes no treino são imputadas pela mediana de cada pipeline, sem consultar teste. Colunas sem observações em todo o treino não são avaliadas no teste. Escores reais ausentes nunca são imputados para avaliar ranking. Correlações indefinidas recebem NaN e um estado explícito; NDCG e regret continuam registrados quando calculáveis. Regret@1 é a melhor acurácia real disponível menos a acurácia do primeiro pipeline recomendado; Regret@5 usa o melhor entre os cinco primeiros. Empates nas predições usam ordem estável das colunas para regret; NDCG usa o tratamento médio de empates da biblioteca.

As médias são calculadas dentro dos grupos e depois entre grupos. Intervalos descritivos de 95% usam 2.000 reamostragens dos grupos com semente 42. Comparações pareadas usam grupos com ambas as métricas definidas. A sobreposição dos treinos limita a interpretação inferencial. O diagnóstico usa cinco correlações exploratórias de Spearman por grupo, com ajuste Benjamini–Hochberg, sem atribuição causal.

3.2.4 Arsenal de Pipelines e Piloto de Hiperparâmetros

O arsenal histórico cruza quatro escalonadores (StandardScaler, MinMaxScaler, RobustScaler e ausência), três seletores (PCA, SelectKBest por ANOVA e ausência) e cinco classificadores. Os seletores usam $q = \min(10, \max(1, P - 1))$. Os classificadores são Random Forest com 100 árvores, SVM RBF com $C = 1$ e gamma scale, k-NN com cinco vizinhos uniformes, MLP com camadas 64 e 32 e até 200 iterações, e árvore de decisão com profundidade máxima 10. Usa-se semente 42 nos algoritmos estocásticos. Demais parâmetros seguem padrões das bibliotecas da execução original; a exportação de parâmetros atuais serve à auditoria, sem presumir versões históricas idênticas.

Manter os 60 candidatos controla o espaço de busca ao comparar metamodelos. Um piloto separado utiliza seis tarefas fixadas em configuração, duas famílias de classificadores e três valores por família, com StandardScaler e sem seletor. Dados brutos usam o alvo da tarefa OpenML; mediana numérica e codificador ordinal categórico são ajustados exclusivamente no treino, e categorias desconhecidas recebem código -1. Os cinco folds estratificados, embaralhados com semente 42, são iguais para todos os valores de uma tarefa. O piloto é exploratório e não substitui uma matriz completa com arsenal expandido.

3.2.5 Reprodutibilidade e Limites da Fase 1

A revisão foi executada com Python 3.14.2, NumPy 2.4.3, pandas 3.0.1, SciPy 1.17.1 e scikit-learn 1.8.0. Manifestos registram configurações, versões e hashes. Splits, predições,

métricas, avisos, falhas e exclusões ficam em arquivos separados; os resultados antigos permanecem fora do diretório da revisão.

A Fase 1 histórica imputou mediana antes da validação e subamostrou globalmente até 5.000 exemplos somente para SVM, de modo que pipelines podem ter sido avaliados em amostras diferentes. A revisão não altera retroativamente esses escores. O piloto novo usa tratamento dentro dos folds; uma reconstrução integral da Fase 1 ainda é necessária para um benchmark com esse tratamento em todas as tarefas.

4 EXPERIMENTOS E RESULTADOS

Este capítulo distingue a reprodução do benchmark histórico da avaliação revisada. Primeiro é auditada a unidade experimental; depois são comparados metamodelos em grupos separados e investigados os erros. Finalmente, um piloto em dados brutos avalia a sensibilidade a hiperparâmetros. As matrizes históricas não foram reconstruídas nesta etapa.

4.1 Auditoria do Meta-conjunto

O acervo contém 201 tarefas OpenML, 186 nomes de conjuntos e 60 pipelines, totalizando 12.060 células, das quais 172 estão ausentes. Tarefas OpenML distintas não garantem conjuntos independentes. A combinação de nomes repetidos e fingerprints de atributos processados identifica 172 grupos conservadores. Todos os tipos de tarefa foram conferidos em metadados oficiais como classificação supervisionada; isso, por si só, não assegura adequação estatística do alvo.

O requisito de pelo menos cinco exemplos em cada classe exclui oito tarefas: 2075, 3486, 362281, 363078, 363094, 363393, 363554 e 363559. A exclusão não utiliza métricas de recomendação. A avaliação principal considera 193 tarefas, 179 nomes e 165 grupos, com 52 escores ausentes. Os critérios e motivos por tarefa constam do catálogo CSV. A análise de sensibilidade com todas as 201 tarefas foi mantida em diretório separado.

A tarefa 363713, anteriormente descrita como holdout, consta da matriz histórica e compartilha o nome do conjunto com outra tarefa. Por isso, a demonstração pontual foi retirada da evidência experimental. Cada task da análise principal passa a receber uma predição de teste em uma partição com grupos separados, conforme o protocolo do capítulo de métodos.

4.2 Reprodução dos Resultados Históricos

A reprodução mantém o preditor original KDTree, as matrizes e os cinco valores de k. Para $k = 10$, o benchmark histórico apresenta Spearman médio de aproximadamente $0{,}5801$, Kendall de $0{,}4767$, Kendall ponderado de $0{,}5222$ e NDCG de $0{,}9844$ sobre 198 tarefas com correlações definidas. Esses números são resultados exploratórios por tarefa, com possibilidade de parentes no treino e seleção de k na própria avaliação; não são a estimativa principal revisada.

Não se atribui toda diferença entre a reprodução e a nova avaliação ao agrupamento: também mudam o número de tarefas de treino por fold, os filtros de qualidade e a unidade usada na agregação. Uma comparação causal exigiria ablações adicionais sob todas as demais condições constantes.

4.3 Comparação em Validação por Grupos

A Tabela 4 apresenta médias calculadas primeiro dentro de cada grupo e depois entre grupos, evitando maior peso de tarefas repetidas. Para as correlações, 192 tarefas em 164 grupos têm valores definidos; a tarefa restante possui alvo constante entre os pipelines observados. NDCG e regret permanecem contabilizados quando definidos, mesmo se a correlação não existe.

Tabela 4 – Avaliação revisada: médias por grupo em cinco partições externas

Método	Spearman	τ_w	NDCG@5	Regret@1
Aleatório	-0.0129	0.0024	0.8687	0.0849
Média global	0.4000	0.3690	0.9396	0.0334
k-NN ajustado	0.4637	0.3995	0.9355	0.0372
k-NN padronizado	0.4736	0.4138	0.9343	0.0339
Random Forest	0.4822	0.4309	0.9414	0.0305

Fonte: Autoria própria (2026), com base nos CSVs da execução revisada..

O k-NN ajustado alcança $\tau_w = 0{,}3995$ e o Random Forest, $0{,}4309$. A diferença pareada média Random Forest menos k-NN é $0{,}0314$, com intervalo descritivo de 95% $[0{,}0073; 0{,}0564]$, estimado por reamostragem dos grupos. A diferença média global menos k-NN é $-0{,}0305$, com intervalo $[-0{,}0637; 0{,}0026]$. Portanto, a vantagem do k-NN sobre a média global não fica estabelecida por esse intervalo. Os resultados sugerem vantagem do Random Forest neste benchmark, sem provar superioridade universal.

O ajuste interno escolheu $k = 10$ em todos os folds externos, de modo que o k-NN fixo em 10 e o ajustado coincidiram nesta execução. A padronização das dimensões foi ajustada exclusivamente no treino. O Random Forest foi utilizado com hiperparâmetros fixos; a comparação não constitui uma busca exaustiva de metamodelos. A Figura 3 mostra a dispersão inferencial descritiva das médias.

4.4 Diagnóstico de Erros e Variabilidade

O k-NN ajustado apresenta 17 tarefas com Kendall ponderado não positivo e 1 com correlação indefinida. Entre os casos de pior ordenação estão task 363159 ($\tau_w = -0{,}5777$); task 9976 ($\tau_w = -0{,}4701$); task 361148 ($\tau_w = -0{,}4383$). O arquivo de diagnóstico associa a cada caso dimensão, tamanho, balanceamento, distância ao vizinho de treino, métricas e estado da avaliação, permitindo inspeção individual.

As correlações exploratórias foram calculadas por grupo para cinco variáveis, com correção Benjamini–Hochberg. O número de atributos tem associação positiva com a qualidade

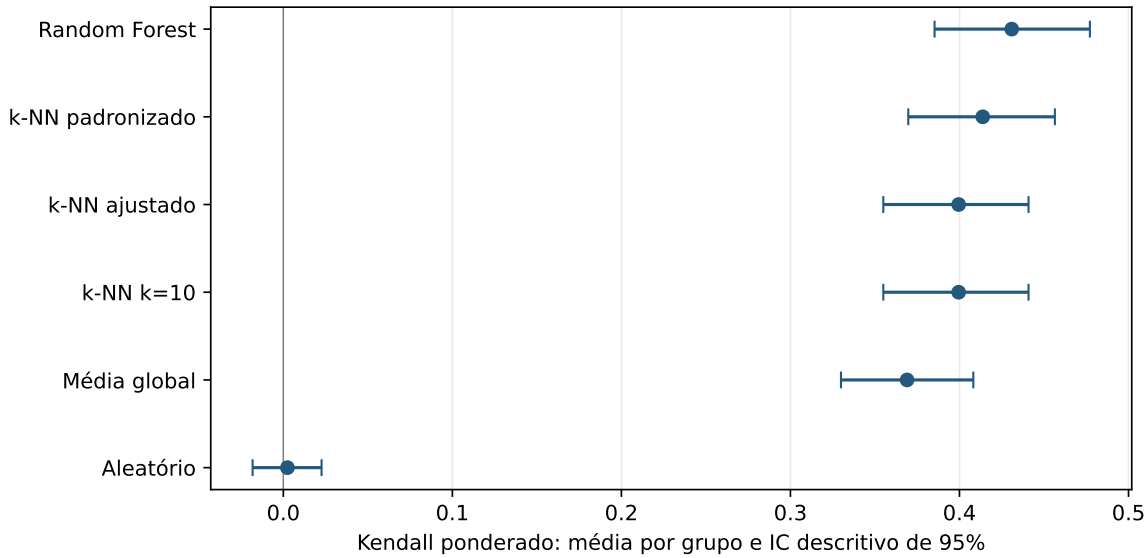


Figura 3 – Kendall ponderado por modelo com intervalos descritivos por grupo
Fonte: Autoria própria (2026)..

($\rho \approx 0{,}2064$, $q \approx 0{,}0400$). As demais variáveis investigadas, incluindo desbalanceamento e distância ao vizinho, não alcançam $q < 0{,}05$ nesta análise. Assim, a inspeção não sustenta atribuir a instabilidade exclusivamente a outliers ou desbalanceamento. A Figura 4 ilustra as relações observadas, sem interpretação causal.

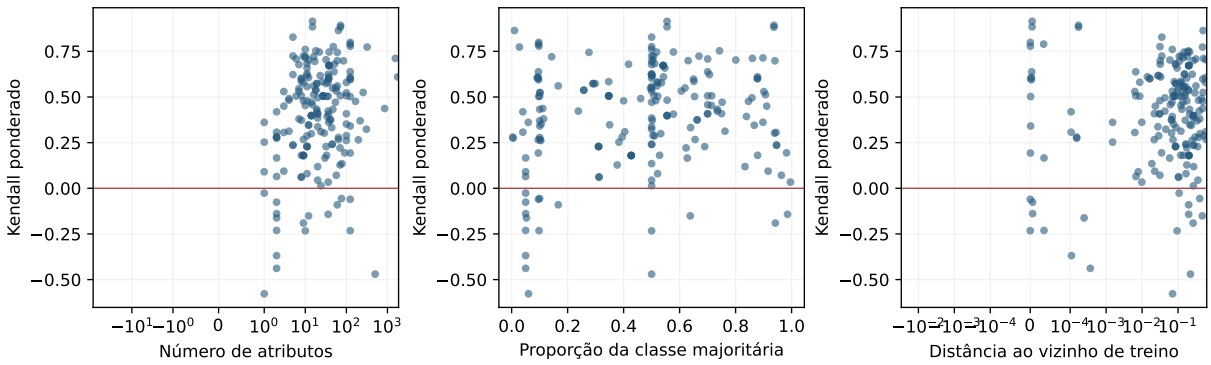


Figura 4 – Diagnóstico do k-NN: dimensionalidade, balanceamento e distância ao treino
Fonte: Autoria própria (2026)..

4.5 Piloto de Variação de Hiperparâmetros

Um experimento separado usa dados brutos de seis tarefas previamente especificadas (11, 15, 37, 4538, 9914 e 361198). Comparam-se profundidades 5, 10 e ilimitada para árvore de decisão e vizinhanças 3, 5 e 11 para k-NN, mantendo StandardScaler e ausência de seletor. Imputação e codificação categórica são ajustadas dentro de cada fold. São 36 combinações de tarefa, classificador e valor, avaliadas nas mesmas cinco divisões por tarefa, totalizando 180 avaliações de treino/teste. Os arquivos preservam índices de linhas, parâmetros, tempos, avisos e resultados por fold.

As amplitudes de acurácia balanceada entre configurações mostram que a escolha de hiperparâmetros pode alterar o desempenho, mas a seleção do melhor valor usando esses próprios folds é exploratória. As seis tarefas foram escolhidas por identificação e diversidade aparente de tamanho e domínio, sem consultar o desempenho de recomendação; não constituem amostra probabilística. O piloto não testa a capacidade de ranqueamento de um arsenal expandido em todos os grupos. Manter os 60 pipelines na avaliação principal controla o espaço de candidatos ao comparar metamodelos. A expansão completa requer nova matriz experimental.

4.6 Interpretação e Limitações

Kendall ponderado avalia concordância de pares com maior peso em itens bem ranqueados; NDCG acumula relevância com desconto logarítmico. Com ganho linear e acurácias próximas, NDCG pode ser alto mesmo com erros na ordenação: a baseline aleatória já alcança NDCG@5 de $0{,}8687$. Logo, NDCG elevado não demonstra acerto dos cinco melhores pipelines. O regret@1 explicita a perda da primeira recomendação, e NDCG@5 limita a avaliação às primeiras posições, complementando as correlações globais.

A representação de 55 descritores e nove zeros oferece uma referência simples e auditável, mas não isola por si só o efeito da representação. Comparar k-NN e Random Forest com o mesmo vetor evidencia dependência do metamodelo; demonstrar ganho da representação exigiria comparar outras representações sob o mesmo protocolo. Tampouco se convertem valores de trabalhos com tarefas diferentes em uma tabela de superioridade: referências recentes orientam o desenho experimental, não fornecem resultados diretamente comparáveis (GIJSBERS *et al.*, 2024; BARATCHI *et al.*, 2024).

Permanecem limitações de origem dos escores: a Fase 1 histórica imputou dados antes da CV e aplicou subamostragem global apenas à SVM. Sua reconstrução completa com tratamento dentro dos folds não foi executada. O agrupamento por nomes e atributos processados reduz repetições observáveis, mas pode não detectar parentes com nomes e transformações diferentes. Os testes são retrospectivos sobre um acervo previamente explorado, sem amostra externa coletada após o desenvolvimento. Os intervalos por bootstrap não incorporam toda a dependência decorrente de folds de treino sobrepostos. Essas restrições delimitam o alcance das conclusões e os requisitos de uma próxima coleta.

5 CONCLUSÃO

Este trabalho desenvolveu e avaliou uma infraestrutura de meta-aprendizado para ranquear pipelines de classificação com descritores estatísticos de conjuntos rotulados. A auditoria e a separação de tarefas relacionadas reformularam a interpretação dos resultados históricos.

5.1 Síntese dos Resultados

O acervo histórico contém 201 tarefas e 60 pipelines, com 172 escores ausentes. A auditoria identificou 186 nomes e 172 grupos conservadores. O requisito de cinco exemplos por classe delimitou a avaliação principal a 193 tarefas em 165 grupos. A tarefa anteriormente apresentada como holdout pertencia ao acervo e foi retirada como evidência de generalização independente.

Em cinco partições externas por grupos, com ajuste interno de k, o Kendall ponderado médio por grupo foi $0{,}3995$ para k-NN, $0{,}4309$ para Random Forest multi-saída e $0{,}3690$ para média global. A diferença pareada Random Forest menos k-NN foi $0{,}0314$, com intervalo descritivo de 95% $[0{,}0073; 0{,}0564]$. A comparação entre k-NN e média global permaneceu inconclusiva pelo intervalo adotado. Os dados sugerem vantagem de Random Forest neste benchmark, sem estabelecer superioridade geral do metamodelo ou da representação.

Foram encontradas 17 tarefas com Kendall ponderado não positivo e uma com correlação indefinida. A associação com número de atributos permaneceu após correção para cinco comparações; desbalanceamento e distância ao treino não explicaram isoladamente a instabilidade. Essas associações não demonstram causalidade.

5.2 Atendimento dos Objetivos

O primeiro objetivo foi atendido pela construção e disponibilização de matrizes, configurações e escores para investigar relações entre características dos dados e desempenho. A auditoria mostrou que parte das tarefas não atendia ao protocolo declarado; sua exclusão documentada integra o resultado científico.

O segundo objetivo foi atendido pela implementação de preditores de ranking e sua comparação com baselines sob grupos separados. O ganho depende do comparador e da métrica. O piloto de hiperparâmetros em dados brutos, com 180 avaliações, demonstrou a viabilidade técnica de ampliar as configurações, mantendo essa evidência separada da matriz histórica.

5.3 Limitações e Perspectivas

A representação determinística permite auditoria: são 55 descritores, nove zeros e normalização L2. As correlações com alvos multiclasses codificados dependem de uma ordenação nominal arbitrária, e o método requer rótulos do problema de entrada. Comparar dois metamodelos com o mesmo vetor não isola o ganho da representação.

A Fase 1 histórica mantém imputação antes da CV e subamostragem global exclusiva da SVM. O piloto novo usa tratamento dentro dos folds, mas não substitui a reconstrução de toda a matriz. Os grupos não garantem independência entre todas as versões dos dados; a avaliação é retrospectiva, Random Forest não foi otimizado e os intervalos de bootstrap são descritivos devido à sobreposição de treinos.

Os próximos passos científicos são: (i) reconstruir o benchmark com dados brutos e pré-processamento dentro dos folds; (ii) avaliar o arsenal expandido sob o mesmo protocolo; (iii) comparar representações estatísticas e neurais com recursos equivalentes; e (iv) validar em conjuntos externos coletados após o desenvolvimento. Isso permitirá distinguir os efeitos da representação, do metamodelo e do espaço de pipelines.

REFERÊNCIAS

ALTMAN, N. S. An introduction to kernel and nearest-neighbor nonparametric regression. **The American Statistician**, v. 46, n. 3, p. 175–185, 1992.

BARATCHI, M. *et al.* Automated machine learning: past, present and future. **Artificial Intelligence Review**, v. 57, p. 122, 2024. Disponível em: https://link.springer.com/article/10.1007/s10462-024-10726-1.

BENTLEY, J. L. Multidimensional binary search trees used for associative searching. **Communications of the ACM**, v. 18, n. 9, p. 509–517, 1975.

BISHOP, C. M. **Pattern Recognition and Machine Learning**. New York: Springer, 2006.

BRAZDIL, P. *et al.* **Metalearning: Applications to Data Mining**. [*S.l.*]: Springer, 2008. (Cognitive Technologies).

BREIMAN, L. Random forests. **Machine Learning**, v. 45, n. 1, p. 5–32, 2001.

BRODERSEN, K. H. *et al.* The balanced accuracy and its posterior distribution. *In*: **Proceedings of the 20th International Conference on Pattern Recognition (ICPR)**. [*S.l.*: *s.n.*], 2010. p. 3121–3124.

COHN, D.; ATLAS, L.; LADNER, R. Improving generalization with active learning. **Machine Learning**, v. 15, p. 201–221, 1994.

CORTES, C.; VAPNIK, V. Support-vector networks. **Machine Learning**, v. 20, n. 3, p. 273–297, 1995.

COVER, T. M.; HART, P. E. Nearest neighbor pattern classification. **IEEE Transactions on Information Theory**, v. 13, n. 1, p. 21–27, 1967.

DOMINGOS, P. **O algoritmo mestre: como a busca pelo algoritmo de machine learning definitivo recriará nosso mundo**. [*S.l.*]: Novatec Editora, 2017.

DOUCET, P. *et al.* Bridging diversity and uncertainty in active learning with self-supervised pre-training. 2025. Disponível em: https://arxiv.org/abs/2403.03728.

DUA, D.; GRAFF, C. **UCI Machine Learning Repository**. 2019. University of California, Irvine, School of Information and Computer Sciences. Disponível em: https://archive.ics.uci.edu/ml.

EDWARDS, H.; STORKEY, A. Towards a neural statistician. *In*: **International Conference on Learning Representations (ICLR)**. [*S.l.*: *s.n.*], 2017.

FEURER, M. *et al.* Efficient and robust automated machine learning. *In*: **Advances in Neural Information Processing Systems 28 (NIPS 2015)**. [*S.l.*]: Curran Associates, Inc., 2015. p. 2962–2970.

FEURER, M.; SPRINGENBERG, J. T.; HUTTER, F. Initializing bayesian hyperparameter optimization via meta-learning. *In*: **Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence**. Austin, Texas, USA: AAAI Press, 2015. p. 1128–1135.

GIJSBERS, P. *et al.* AMLB: an AutoML benchmark. **Journal of Machine Learning Research**, v. 25, n. 101, p. 1–65, 2024. Disponível em: https://www.jmlr.org/papers/v25/22-0493.html.

GOLDBLUM, M. *et al.* Position: The no free lunch theorem, kolmogorov complexity, and the role of inductive biases in machine learning. *In*: **Proceedings of the 41st International Conference on Machine Learning**. PMLR, 2024. (Proceedings of Machine Learning Research, v. 235), p. 15788–15808. Disponível em: https://proceedings.mlr.press/v235/goldblum24a.html.

GOLSHANRAD, P. *et al.* Mega: Predicting the best classifier combination using meta-learning and a genetic algorithm. **Intelligent Data Analysis**, v. 25, n. 6, p. 1547–1563, 2021. Disponível em: https://journals.sagepub.com/doi/abs/10.3233/IDA-205494.

GUYON, I.; ELISSEEFF, A. An introduction to variable and feature selection. **Journal of Machine Learning Research**, v. 3, p. 1157–1182, 2003.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural Computation**, v. 9, n. 8, p. 1735–1780, 1997.

HOLLMANN, N. *et al.* Accurate predictions on small data with a tabular foundation model. **Nature**, v. 637, p. 319–326, 2025. Disponível em: https://doi.org/10.1038/s41586-024-08328-6.

JOLLIFFE, I. T. **Principal Component Analysis**. 2. ed. New York: Springer, 2002.

JOMAA, H. S.; SCHMIDT-THIEME, L.; GRABOCKA, J. Dataset2vec: Learning dataset meta-features. **Data Mining and Knowledge Discovery**, v. 35, n. 3, p. 964–985, 2021.

KENDALL, M. G. A new measure of rank correlation. **Biometrika**, v. 30, n. 1/2, p. 81–93, 1938.

KHAN, I. *et al.* Autofe-sel: A meta-learning based methodology for recommending feature subset selection algorithms. **KSII Transactions on Internet and Information Systems**, v. 17, p. 1773–1793, 07 2023.

LIPPMANN, R.; CAMPBELL, W.; CAMPBELL, J. An overview of the DARPA data driven discovery of models (D3M) program. *In*: **29th Conference on Neural Information Processing Systems (NIPS)**. Barcelona, Spain: [*s.n.*], 2016.

MAHER, M.; SAKR, S. SmartML: A meta learning-based framework for automated selection and hyperparameter tuning for machine learning algorithms. *In*: **Proceedings of the 22nd International Conference on Extending Database Technology (EDBT 2019)**. Lisboa, Portugal: [*s.n.*], 2019. p. 554–557.

OLSON, R. S. *et al.* Automating biomedical data science through tree-based pipeline optimization. *In*: **Applications of Evolutionary Computation**. [*S.l.*]: Springer, 2016. p. 123–137.

PEDREGOSA, F. *et al.* Scikit-learn: Machine learning in python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.

PENG, H. A comprehensive overview and survey of recent advances in meta-learning. **CoRR**, abs/2004.11149, 2020. Disponível em: https://arxiv.org/abs/2004.11149.

PEREIRA, G. T.; SANTOS, M. R. dos; CARVALHO, A. C. P. de Leon Ferreira de. Evaluating meta-feature selection for the algorithm recommendation problem. **CoRR**, abs/2106.03954, 2021. Disponível em: https://arxiv.org/abs/2106.03954.

QUINLAN, J. R. Induction of decision trees. **Machine Learning**, v. 1, n. 1, p. 81–106, 1986.

RAMAMURTHY, K. N.; VARSHNEY, K. R.; MODY, K. **Topological Data Analysis of Decision Boundaries with Application to Model Selection**. 2018. Disponível em: https://arxiv.org/abs/1805.09949.

REIF, M.; SHAFAIT, F.; DENGEL, A. Meta2-features: Providing meta-learners more information. *In*: **Poster and Demo Track of the 35th German Conference on Artificial Intelligence (KI)**. [*S.l.*: *s.n.*], 2012.

SCHOENFELD, B.; SEPPI, K.; GIRAUD-CARRIER, C. Metalearning using structure-rich pipeline representations for improved automl. **International Journal of Data Analysis Techniques and Strategies**, v. 14, n. 1, p. 49–72, 2022. Disponível em: https://www.inderscienceonline.com/doi/abs/10.1504/IJDATS.2022.129174.

SCHOENFELD, B. J. **Metalearning by Exploiting Granular Machine Learning Pipeline Metadata**. 2020. Dissertação (Mestrado) — Brigham Young University, 2020. Disponível em: https://scholarsarchive.byu.edu/etd/8730/.

SENER, O.; SAVARESE, S. Active learning for convolutional neural networks: A core-set approach. *In*: **International Conference on Learning Representations (ICLR)**. [*S.l.*: *s.n.*], 2018.

SETTLES, B. **Active Learning Literature Survey**. [*S.l.*], 2009. Disponível em: https://minds.wisconsin.edu/bitstream/handle/1793/60660/TR1648.pdf?sequence=1&isAllowed=y.

SPEARMAN, C. The proof and measurement of association between two things. **The American Journal of Psychology**, v. 15, n. 1, p. 72–101, 1904.

TRIANTAFILLOU, E. *et al.* Meta-dataset: A dataset of datasets for learning to learn from few examples. *In*: **International Conference on Learning Representations (ICLR)**. [*s.n.*], 2020. Disponível em: https://arxiv.org/abs/1903.03096.

VANSCHOREN, J. Meta-learning: A survey. **arXiv preprint arXiv:1810.03548**, 2018. Disponível em: https://arxiv.org/abs/1810.03548.

VANSCHOREN, J. *et al.* Openml: Networked science in machine learning. **ACM SIGKDD Explorations Newsletter**, v. 15, n. 2, p. 49–60, 2014. Disponível em: https://www.kdd.org/exploration_files/15-2-2013-12.pdf#page=51.

VARSHNEY, K. R.; RAMAMURTHY, K. N. Persistent topology of decision boundaries. **ICASSP**, IEEE, p. 3931–3935, 2015.

VILALTA, R.; DRISSI, Y. A perspective view and survey of meta-learning. **Artificial Intelligence Review**, v. 18, p. 77–95, 2002.

WANG, G.; SONG, Q.; ZHU, X. **Ensemble Learning Based Classification Algorithm Recommendation**. 2021. Disponível em: https://arxiv.org/abs/2101.05993.

WISTUBA, M.; SCHILLING, N.; SCHMIDT-THIEME, L. Two-stage transfer surrogate model for automatic hyperparameter optimization. *In*: **Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML/PKDD)**. [*S.l.*: *s.n.*], 2016. p. 13–28.

WOLPERT, D. H.; MACREADY, W. G. No free lunch theorems for optimization. **IEEE Transactions on Evolutionary Computation**, v. 1, n. 1, p. 67–82, 1997.